

Foundations of Retrieval-Augmented Generation

How AI assistants answer from enterprise data: embeddings, vector search and grounded responses

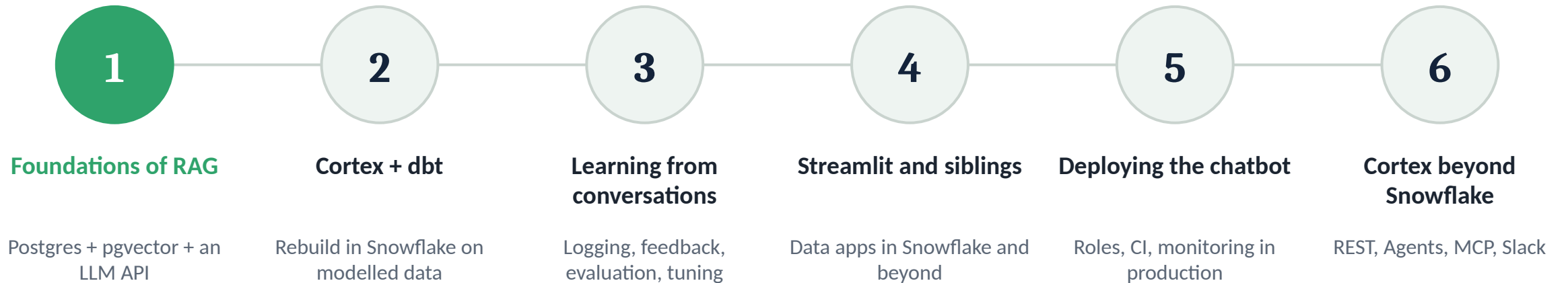
R. Firman Insan Muhammad (Vim)

SnowPro Certified · Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



One chatbot, six iterations



Today's output: a working baseline chatbot and a small golden test set. Every later module is measured against it.

By the end of today you can...



Explain

why RAG exists: parametric vs non-parametric knowledge, and the failure modes retrieval addresses.



Reason

about embeddings, similarity metrics and chunking as design decisions with trade-offs, not library defaults.



Compare

ANN index families (HNSW, IVF, PQ) and choose a vector store for a given workload and organisation.



Build and critique

an end-to-end RAG chatbot over Postgres + pgvector, and measure its retrieval quality.

An LLM alone cannot answer questions about your business



Stale

Knowledge freezes at the training cutoff. Last week's reward change or a member's points balance simply does not exist for the model.



Blind

Your tables, documents and tickets were never in the training data, and should not be. The model has no access path to them.



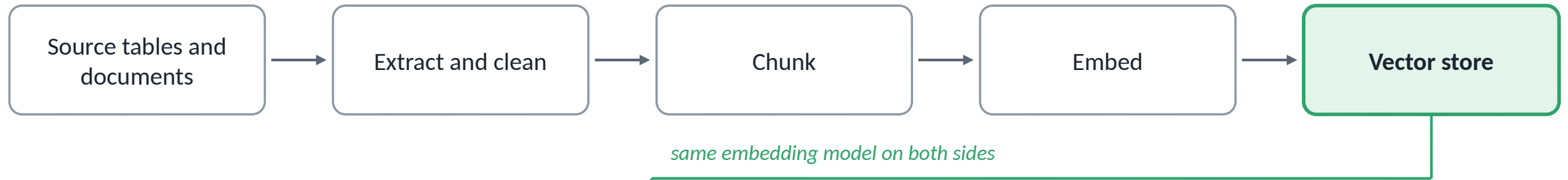
Unaccountable

Answers arrive fluent and confident with no provenance. You cannot tell recall from invention, and neither can a regulator.

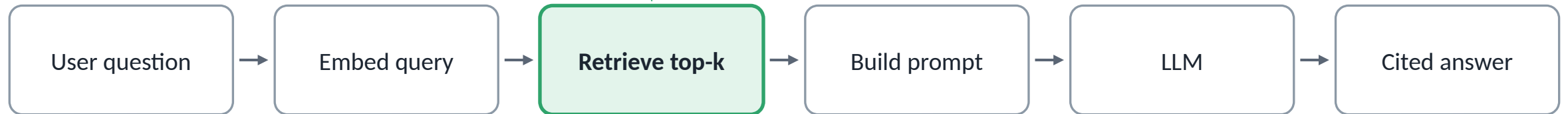
RAG in one sentence: *give the model an open book at inference time, and make it show which page it read.*

RAG is two pipelines that share one index

OFFLINE • INDEXING (BATCH OR CDC)



ONLINE • PER QUESTION (LATENCY-SENSITIVE)



Data engineering owns the top lane. Most production RAG failures start there.

Where RAG came from



Today's lab is "naive RAG" in the survey's taxonomy. Modules 2 and 3 move us to advanced RAG.

01

Representing meaning

Embeddings, similarity and chunking: the three decisions that fix retrieval quality before any LLM is called

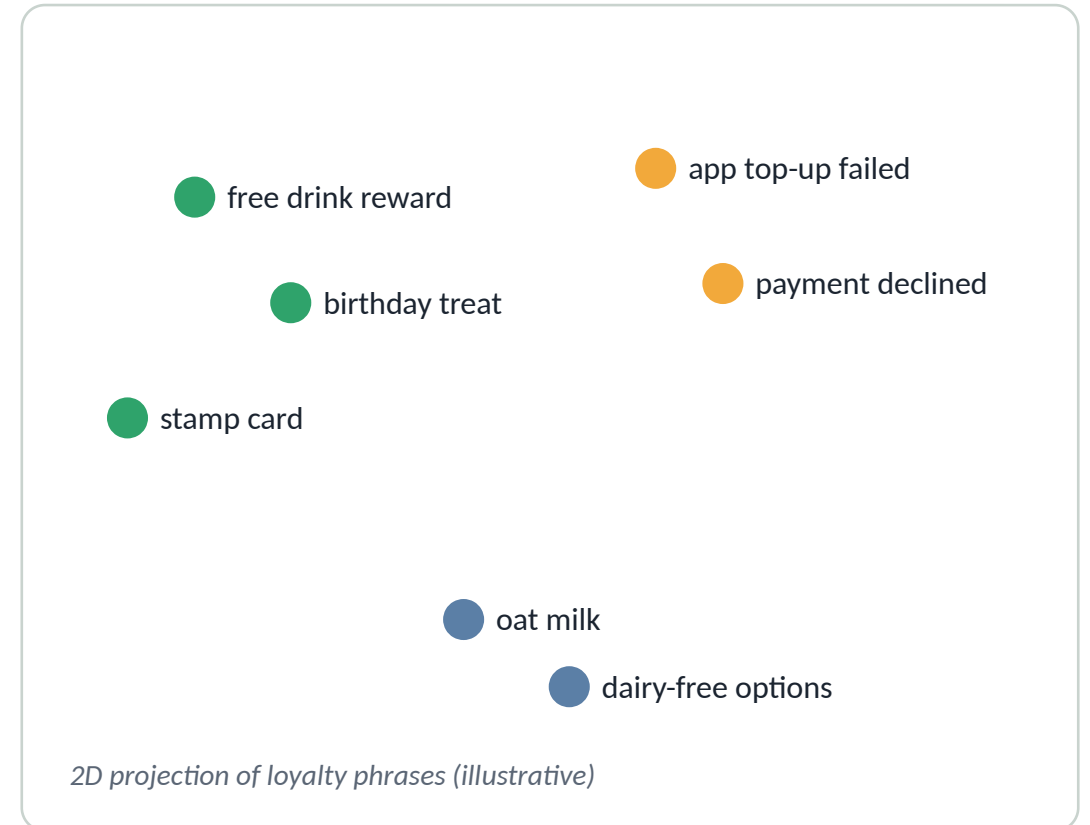
Text becomes a point; meaning becomes distance

An embedding model f maps text to a vector in $\mathbb{R}^d$ so that similar meaning implies nearby vectors.

$$\cos(q, d) = (q \cdot d) / (\|q\| \|d\|)$$

Metric	Measures	pgvector op
Cosine distance	Angle only	<=>
Inner product	Angle and magnitude	<#>
Euclidean (L2)	Straight-line distance	<->

Normalise to unit length and all three rank identically.



Choose the model on your queries, not the leaderboard

Model	Dims	Max tokens	Why consider it
all-MiniLM-L6-v2	384	256	Tiny, fast English baseline
BAAI/bge-small-en-v1.5	384	512	Today's lab default: strong for its size
BAAI/bge-m3	1024	8192	Multilingual; dense and sparse outputs
OpenAI text-embedding-3-small	1536	8191	Hosted API; dimensions can be shortened
Snowflake arctic-embed-m-v1.5	768	512	Open weights; available inside Cortex (L2)

Rule: never mix embedding models in one index. The model is part of the schema; changing it means re-embedding everything.

Selection criteria

- Languages in your corpus and queries
- Domain vocabulary (menu, loyalty terms)
- Max input vs your chunk size
- Dimensions: storage and latency cost
- Licence and data residency
- Recall on your own golden set

The most underrated decision in the pipeline

Fixed-size + overlap

Every N tokens, with an overlap window

amber = overlap



Structural (recursive)

Split on headings, then paragraphs, then sentences



Semantic

Split where adjacent-sentence similarity drops



Parent-child

Match small chunks, return their parent section



Too small: precise hits, missing context. **Too large:** diluted embedding, wasted prompt tokens.

Starting point: 300 to 800 tokens, 10 to 20% overlap, respect document structure. Then measure.

02

Vector indexes and stores

Why exact search does not scale, how approximate nearest-neighbour indexes work, and where to keep your vectors

Exact search is linear; production needs approximate

$O(n \cdot d)$

per query for exact search

10M chunks $\times$ 768 dims $\approx$ 7.7 billion
multiply-adds for every single question



HNSW (graph)

Navigable multi-layer graph. High recall, fast queries, incremental inserts. Memory-hungry.



IVF (clustering)

k-means partitions; search the nprobe nearest lists. Needs training; suits large, stable sets.



PQ (compression)

Split vectors into sub-vectors and store codes. Big memory savings, lower recall. Often IVF-PQ.

Skip-list thinking in high-dimensional space



Parameter	Trades	pgvector default
m	Links per node: memory vs recall	16
ef_construction	Build time vs graph quality	64
ef_search	Query latency vs recall	40

Greedy descent: find the nearest node on each layer, then drop down. Tune ef_search per query, not per index.

Extend the database you run before adding a new one



Extension of an existing DB

pgvector, Snowflake VECTOR,
Elasticsearch / OpenSearch, MongoDB
Atlas

*Pick when data already lives there and
governance matters*



Purpose-built vector DB

Pinecone, Qdrant, Weaviate, Milvus

*Pick at very large scale or high QPS, or for
their advanced features*



Library you embed

FAISS, hnswlib, ScaNN

*Pick for research, benchmarks, or when you
own serving end to end*

Data engineer's lens: freshness, joins with metadata, one access-control model, one fewer system to run.

03

Reading from the database

Two kinds of question, two retrieval routes, and the lab schema that ties them together

Similarity search cannot count



Semantic: RAG over text

"Can members redeem a free drink on a delivery order?"

Sources: programme terms, rewards FAQs, store guides

Mechanism: embed, retrieve top-k, ground the answer



router



Analytical: text-to-SQL

"How many free-drink rewards were redeemed in London in Q2?"

Sources: members, rewards, transactions tables

Mechanism: generate SQL over a modelled schema, execute, explain

Module 2 preview: Cortex Search is the managed left route, Cortex Analyst the managed right route, both fed by dbt models.

Keep chunks beside their source key

POSTGRES 16 + PGVECTOR

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE doc_chunks (
  id          BIGSERIAL PRIMARY KEY,
  doc_id      TEXT NOT NULL,
  chunk_no    INT  NOT NULL,
  chunk_text  TEXT NOT NULL,
  metadata    JSONB,    -- channel, section, model
  embedding   VECTOR(384),
  updated_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE (doc_id, chunk_no)
);

CREATE INDEX ON doc_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

CREATE INDEX ON doc_chunks USING gin (metadata);
```

Lineage

doc_id ties every chunk to its source row or file, so deletes and updates propagate.

Idempotency

(doc_id, chunk_no) is the upsert key; reruns never duplicate.

Pre-filtering

metadata carries channel, section and model version for WHERE clauses.

Fixed dimension

384 is set by the embedding model; a new model means a new column or table.

Ingestion: chunk, embed, upsert

INGEST.PY

```
from sentence_transformers import SentenceTransformer
from pgvector.psycopg import register_vector
from psycopg.types.json import Jsonb
import psycopg

MODEL = "BAAI/bge-small-en-v1.5"          # 384 dims
model = SentenceTransformer(MODEL)

def chunk(text, size=800, overlap=150):
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]

UPSERT = """INSERT INTO doc_chunks
(doc_id, chunk_no, chunk_text, metadata, embedding)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (doc_id, chunk_no) DO UPDATE
SET chunk_text = EXCLUDED.chunk_text,
    embedding = EXCLUDED.embedding, updated_at = now()"""

with psycopg.connect(DSN) as conn:
    register_vector(conn)
    for doc in load_loyalty_docs():          # id, text, meta
        parts = chunk(doc.text)
        vecs = model.encode(parts, normalize_embeddings=True)
        for i, (t, v) in enumerate(zip(parts, vecs)):
            conn.execute(UPSERT, (doc.id, i, t,
                                   Jsonb(**doc.meta, "model": MODEL}), v))
```

Production gaps to close

- Chunk by structure, not characters
- Batch inserts; do not commit per row
- Delete orphaned chunks when a document shrinks
- Re-embed only changed documents
- Log counts and failures per run

Retrieve, then generate with citations

RETRIEVE: RAG.PY

```
def retrieve(question, k=5, channel=None):
    qv = model.encode(question,
                        normalize_embeddings=True)
    return conn.execute("""
        SELECT doc_id, chunk_no, chunk_text,
               1 - (embedding <=> %s) AS score
        FROM doc_chunks
        WHERE %s::text IS NULL
           OR metadata->>'channel' = %s
        ORDER BY embedding <=> %s
        LIMIT %s""",
        (qv, channel, channel, qv, k)).fetchall()
```

GENERATE: RAG.PY

```
SYSTEM = """You are Neo Nexus, the loyalty
assistant. Answer ONLY from the
context. Cite sources as [doc_id#chunk].
If the context lacks the answer, say so."""

def answer(question, channel=None):
    hits = retrieve(question, 5, channel)
    context = "\n\n".join(
        f"[{h[0]},{h[1]}] {h[2]}" for h in hits)
    user = f"Context:\n{context}\n\n" \
          f"Question: {question}"
    return llm_chat(SYSTEM, user, hits)
```

Same model, both sides

The query is embedded exactly as the chunks were, normalised.

IDs make answers checkable

Every citation maps back to one row in doc_chunks.

Abstaining beats guessing

An honest "I don't know" is a correct answer.

One interface, any engine: Claude, GPT or Llama

LLM.PY

```
# llm.py: rag.py only ever calls llm_chat(system, user)
def llm_chat(system, user, engine="claude"):
    if engine == "claude":                # Anthropic API
        import anthropic
        r = anthropic.Anthropic().messages.create(
            model="claude-sonnet-5-5", max_tokens=1024,
            system=system,
            messages=[{"role": "user", "content": user}])
        return r.content[0].text
    if engine == "openai":                # OpenAI API
        from openai import OpenAI
        r = OpenAI().chat.completions.create(
            model="gpt-5-mini",
            messages=[{"role": "system", "content": system},
                      {"role": "user", "content": user}])
        return r.choices[0].message.content
    if engine == "llama":                 # local, via Ollama
        import ollama
        r = ollama.chat(model="llama3.3:70b", messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user}])
        return r["message"]["content"]
```

Anthropic Claude

API key · data goes to Anthropic

OpenAI GPT

API key · data goes to OpenAI

Meta Llama

open weights · runs on your hardware

Design rule: keep the engine behind one function so it can be swapped and compared.

Size the engine to the job, then check where the data goes

Engine family	Tiers (large → small)	How you access it	Where the prompt goes	Typical fit
Anthropic Claude	Opus → Sonnet → Haiku	Anthropic API; major clouds	Anthropic, or your cloud provider	Long documents, careful reasoning
OpenAI GPT	GPT-5 → mini → nano	OpenAI API; Azure OpenAI	OpenAI or Azure	Broad ecosystem and tooling
Meta Llama	70B → 8B	Open weights: Ollama, vLLM, clouds	Can stay on your hardware	Privacy, offline, cost at scale
Mistral	Large → small	API or open weights	Vendor or self-hosted	European hosting, efficient models

Small, fast

routing, classification, query rewriting

Mid-tier

grounded answers from retrieved context

Largest

hard reasoning, evaluation judging

For RAG, context quality usually matters more than engine size. Measure on your golden set before paying for the biggest model.

The prompt is a contract, not a wish

Role and rules

Who the assistant is; answer only from context

Retrieved context

Chunks, each tagged with a stable ID

User question

Verbatim, after the context

Output contract

Cite IDs; abstain if unsupported; format

Lost in the middle

Models use the start and end of long contexts best. Rank the strongest chunks first and keep k small.

Context budget

Beyond a few relevant chunks, extra context adds cost and distraction, not accuracy.

Indirect prompt injection

Retrieved text is untrusted input. A document can contain instructions; treat it as data.

Four upgrades, each fixing a named failure



Hybrid search

Fixes: exact terms (reward codes, clause numbers) lost by dense vectors. BM25 + vector, fused with Reciprocal Rank Fusion.



Reranking

Fixes: plausible but wrong top hits. A cross-encoder rescores the top 50 and keeps the best 5.



Query transformation

Fixes: vague or multi-part questions. Rewrite, split into sub-queries, or embed a hypothetical answer (HyDE).



Structure and metadata

Fixes: right topic, wrong channel or version. Pre-filter, parent-child retrieval, graph-based retrieval.

If you cannot measure retrieval, you are guessing



Retrieval quality

Recall@k: is a correct chunk in the top k?

MRR: how high is the first correct chunk?

nDCG: graded relevance, discounted by rank

Needs labelled question-to-chunk pairs



Answer quality

Faithfulness: is every claim supported by context?

Answer relevance: does it address the question?

Context precision: was the retrieved context useful?

Usually LLM-as-judge with a rubric (e.g. RAGAS)

Today: a 20-question golden set and recall@5. **Module 3:** the full evaluation and feedback loop.

Build the Neo Nexus loyalty assistant

- 1 Start Postgres + pgvector**
docker run pgvector/pgvector:pg16; create schema
10 min
- 2 Load sample data**
loyalty tables and 12 programme documents
10 min
- 3 Chunk and embed twice**
two configs: 400 and 1,000 characters
25 min
- 4 Retrieve and answer**
rag.py with citations and an abstain rule
25 min
- 5 Evaluate**
20-question golden set; recall@5 per config
20 min

Deliverables

- Repo: ingest.py, rag.py, eval notebook
- Recall@5 for both chunk sizes
- Three failure cases, each diagnosed: retrieval or generation?
- One-page reflection: which chunk size, and why

Stretch: structural chunking, or BM25 + RRF hybrid

Questions worth arguing about

1. With million-token context windows, when is retrieval still worth building?
2. Is cosine similarity a good proxy for relevance? Construct a counter-example.
3. Where should access control live in a RAG system: the index, the query, or the prompt?
4. What does freshness mean for an index when the source table changes hourly?



Reading list

Lewis et al. (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Karpukhin et al. (2020) Dense Passage Retrieval for Open-Domain QA. EMNLP.

Malkov and Yashunin (2018) Efficient and robust ANN search using HNSW graphs. IEEE TPAMI.

Liu et al. (2023) Lost in the Middle: How Language Models Use Long Contexts.

Gao et al. (2023) Retrieval-Augmented Generation for LLMs: A Survey.

Es et al. (2023) RAGAS: Automated Evaluation of RAG.

Same chatbot, rebuilt inside Snowflake



dbt layer

Staging and mart models, tested and documented, instead of raw tables



Cortex Search

Replaces today's embedding code, index and hybrid search



Cortex Analyst

Answers the analytical questions via a semantic model

Before Module 2: finish the lab, read Lewis et al. (2020) and Liu et al. (2023), and create a Snowflake trial account in a Cortex-enabled region.

Snowflake Cortex on a Governed Data Foundation

Rebuilding the assistant inside the Data Cloud: dbt-modelled data, Cortex Search for documents and Cortex Analyst for metrics

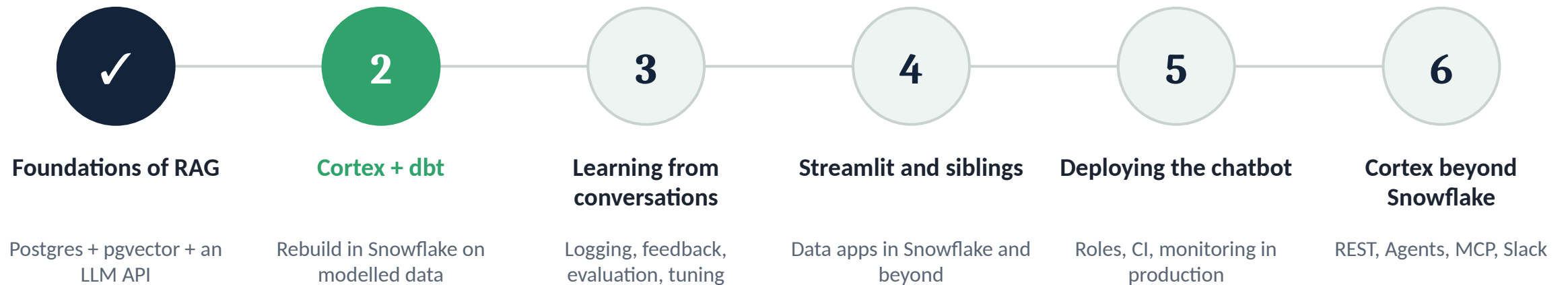
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



Same assistant, now inside the Data Cloud



Today's output: a dbt project, a Cortex Search service, a semantic view, and a recall@5 comparison against the Module 1 baseline.

By the end of today you can...



Map

each hand-built Module 1 component to its Snowflake Cortex equivalent, and explain what you give up.



Design

a dbt layer (staging, intermediate, marts) that makes both documents and tables AI-ready.



Build

a Cortex Search service on modelled chunks and a semantic view that Cortex Analyst can query.



Govern

the assistant with roles, masking and row access, and compare it fairly with the Module 1 baseline.

Everything we built by hand has a Snowflake equivalent

Module 1 component	Module 2 equivalent	Where it runs
PDF extraction in Python	AI_PARSE_DOCUMENT	dbt staging model
Character chunking	SPLIT_TEXT_RECURSIVE_CHARACTER	dbt intermediate model
Embedding model + pgvector HNSW	Cortex Search service	Serving layer
Vector search only	Hybrid: vector + keyword + semantic reranking	Built into Cortex Search
External LLM API call	AI_COMPLETE (Cortex LLM functions)	Inside Snowflake
<i>Not handled</i>	Cortex Analyst over a semantic view	Serving layer
Cron job re-runs ingest	TARGET_LAG + dbt schedule	Declarative

Green rows are new capabilities, not just replacements.

An LLM reads exactly what you give it

RAW.REWARDS_EXPORT

RWD_ID	RWD_STS	AMT	ISS_DT
RW-0091	R	3.40	20260411
RW-0091	R	3.40	20260411
RW-0107	I		11/04/2026

dbt

MARTS.FCT_REWARDS

reward_id	reward_status	reward_value	issued_date
RW-0091	Redeemed	3.40	2026-04-11
RW-0107	Issued	null	2026-04-11

Duplicates

double loads inflate every count

Cryptic codes

the model must guess that R means Redeemed

Mixed formats

dates and amounts parse inconsistently

No declared grain

one row per what? The LLM cannot know

Principle: the semantic layer an AI reads is a product. Model it, test it and document it like one.

01

The Cortex toolkit

What Snowflake manages for you: AI SQL functions, Cortex Search, Cortex Analyst and agents, all inside the governance boundary

Four capabilities on one governance foundation



AI SQL functions

AI_COMPLETE,
AI_PARSE_DOCUMENT,
AI_CLASSIFY, AI_EXTRACT,
AI_EMBED

*Row-level building blocks, callable
in dbt*



Cortex Search

Managed hybrid retrieval over
text with incremental refresh

Replaces Module 1's vector store



Cortex Analyst

Text-to-SQL grounded in a
semantic view

Answers the analytical questions



Cortex Agents

Plan, call Search and Analyst as
tools, and respond

Powers Snowflake Intelligence



Governance foundation: data stays in Snowflake; roles, masking and row access policies apply to every capability above.

One statement replaces Module 1's whole top lane

SNOWFLAKE SQL

```
CREATE OR REPLACE CORTEX SEARCH SERVICE
  serving.loyalty_search
  ON chunk_text
  ATTRIBUTES channel, doc_id
  WAREHOUSE = transform_wh
  TARGET_LAG = '1 hour'
  EMBEDDING_MODEL = 'snowflake-arctic-embed-l-v2.0'
AS
  SELECT chunk_id, doc_id, channel, chunk_text
  FROM marts.doc_chunks;
```

Requires change tracking on the source table.

ON

the text column that is searched

ATTRIBUTES

filterable columns, like last week's metadata WHERE clause

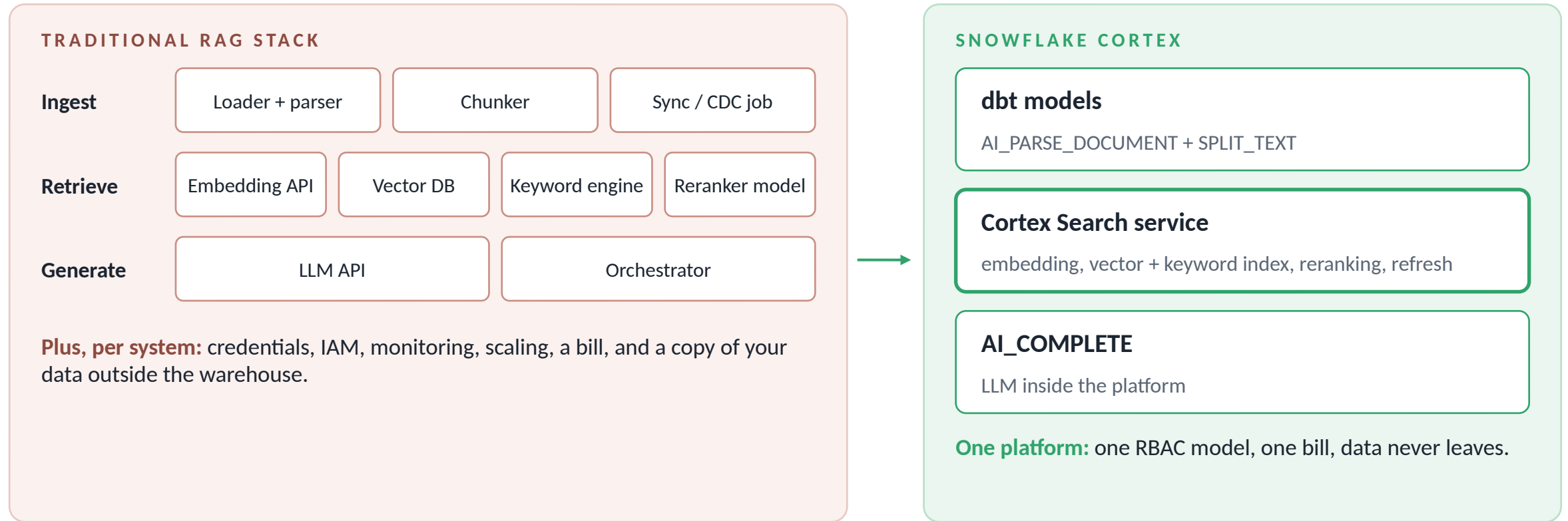
TARGET_LAG

declared freshness; incremental refresh

Hybrid by default

vector + keyword + semantic reranking

Nine moving parts collapse into three objects



9 → 3

moving parts to build and run

Many → 1

vendors, bills and security boundaries

Scripted → declared

freshness via TARGET_LAG

The operational burden moves to the platform

Task	Traditional RAG: you build and run it	Cortex: the platform does it
Keep index in sync with source	CDC job or cron, backfills, drift checks	TARGET_LAG incremental refresh
Embed chunks and queries identically	Same model in two code paths	Automatic, one model per service
Hybrid (vector + keyword) search	Run a second engine, fuse the scores	Built in
Reranking	Host and scale a cross-encoder	Built in
Scale and tune the index	Shard, tune HNSW, plan capacity	Managed serving
Change embedding model	Re-embed, dual-write, cut over	Recreate the service (still a full re-index)
Access control	Mirror permissions in every system	Snowflake roles on the service
Data residency	Text and vectors copied to vendors	Stays in Snowflake

Still your job: data modelling, chunking choices, prompt design and evaluation. Cortex simplifies infrastructure, not judgement.



Code, before and after

Seven stages of the same assistant, written the traditional way and the Cortex way, side by side

PDF parsing and chunking: a script becomes one query

TRADITIONAL RAG • PYTHON

16 lines

```
from pathlib import Path
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1800, chunk_overlap=300,
    separators=["\n## ", "\n\n", "\n", " "])

def parse_and_chunk(path, channel):
    reader = PdfReader(path)
    text = "\n".join(p.extract_text() or "" for p in reader.pages)
    text = strip_headers_footers(text) # your own helper
    return [{"doc_id": path.name, "channel": channel,
            "chunk_no": i, "chunk_text": c}
            for i, c in enumerate(splitter.split_text(text))]

for path in Path("loyalty_docs").rglob("*.pdf"):
    if path.stat().st_mtime > last_run_ts(): # incremental
        write_chunks(parse_and_chunk(path, path.parent.name))
```

CORTEX • SQL (DBT MODEL)

14 lines

```
select relative_path      as doc_id,
       split_part(relative_path,
                   '/', 1) as channel,
       c.index            as chunk_no,
       c.value::string    as chunk_text
from directory(@raw.loyalty_docs) d,
lateral flatten(input =>
  snowflake.cortex
    .split_text_recursive_character(
      ai_parse_document(
        to_file('@raw.loyalty_docs',
                d.relative_path),
        {'mode': 'LAYOUT'}):content::string,
        'markdown', 1800, 300)) c
```

Takeaway: no PDF library, no splitter dependency, no state file for incremental runs; dbt's incremental config replaces last_run_ts().

Embedding and two indexes: a pipeline becomes a declaration

TRADITIONAL RAG • PYTHON

17 lines

```
from sentence_transformers import SentenceTransformer
from elasticsearch import Elasticsearch, helpers

model = SentenceTransformer("BAAI/bge-small-en-v1.5")
es = Elasticsearch(ES_URL, api_key=ES_KEY)

def index_chunks(rows):
    vecs = model.encode([r["chunk_text"] for r in rows],
                        normalize_embeddings=True, batch_size=64)
    with pg.cursor() as cur:
        # vector index
        for r, v in zip(rows, vecs):
            cur.execute(UPSERT_SQL, (r["doc_id"], r["chunk_no"],
                                     r["chunk_text"], Jsonb(r), v))
    helpers.bulk(es, ({"_index": "loyalty_chunks", # keyword index
                      "_id": f'{r["doc_id"]}{r["chunk_no"]}',
                      "_source": r} for r in rows))

    pg.commit()

# not shown: HNSW DDL, ES mapping, orphan deletes,
# keeping both indexes consistent, re-embedding on model change
```

CORTEX • SQL

10 lines

```
create cortex search service
    serving.loyalty_search
    on chunk_text
    attributes channel, doc_id
    warehouse = transform_wh
    target_lag = '1 hour'
as
select chunk_id, doc_id,
       channel, chunk_text
from marts.doc_chunks;
```

Takeaway: one statement owns the embedding model, the vector index, the keyword index and keeping both in sync.

Hybrid retrieval and reranking: a module becomes a call

TRADITIONAL RAG • PYTHON

20 lines

```
from collections import defaultdict
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-base")

def search(q, channel, k=5, n=50):
    qv = model.encode(q, normalize_embeddings=True)
    dense = pg.execute(DENSE_SQL,
                      (qv, channel, channel, qv, n)).fetchall()
    sparse = es.search(index="loyalty_chunks", size=n, query={
        "bool": {"must": {"match": {"chunk_text": q}},
        "filter": {"term": {"channel": channel}}}}
    )["hits"]["hits"]
    scores = defaultdict(float) # reciprocal rank fusion
    for rank, h in enumerate(dense):
        scores[h.chunk_id] += 1 / (60 + rank)
    for rank, h in enumerate(sparse):
        scores[h["_id"]] += 1 / (60 + rank)
    fused = sorted(scores, key=scores.get, reverse=True)[:n]
    texts = fetch_texts(fused)
    rr = reranker.predict([(q, t) for t in texts])
    return [c for _, c in sorted(zip(rr, fused), reverse=True)][:k]
```

CORTEX • PYTHON

8 lines

```
hits = svc.search(
    query=q,
    columns=["chunk_id",
             "chunk_text"],
    filter={"@eq":
            {"channel": channel}},
    limit=5,
).results
```

Takeaway: dense search, keyword search, fusion and reranking all happen server-side in one call.

Calling the LLM: a client with retries becomes a function

TRADITIONAL RAG • PYTHON

16 lines

```
import os, time
from openai import OpenAI, RateLimitError

client = OpenAI(api_key=os.environ["LLM_API_KEY"])

def generate(prompt, retries=3):
    for attempt in range(retries):
        try:
            r = client.chat.completions.create(
                model=LLM_MODEL,
                messages=[{"role": "user", "content": prompt}],
                temperature=0)
            return r.choices[0].message.content
        except RateLimitError:
            time.sleep(2 ** attempt)
    raise RuntimeError("LLM unavailable")

# not shown: secret rotation, egress approval,
# vendor DPA, per-vendor usage tracking
```

CORTEX • SQL

4 lines

```
select ai_complete(
    'claude-sonnet-4-5',
    :prompt
) as answer;
```

Takeaway: no API keys, no retry loop, no data leaving the platform; usage is metered with everything else.

Freshness: an orchestration DAG becomes one parameter

TRADITIONAL RAG • PYTHON

16 lines

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

with DAG("loyalty_rag_refresh", schedule="@hourly",
        start_date=datetime(2026, 1, 1), catchup=False) as dag:
    detect = PythonOperator(task_id="detect_changed_docs",
                           python_callable=detect_changes)
    parse  = PythonOperator(task_id="parse_and_chunk",
                           python_callable=run_parse)
    embed  = PythonOperator(task_id="embed_and_index",
                           python_callable=run_index)
    prune  = PythonOperator(task_id="delete_orphans",
                           python_callable=prune_deleted)
    verify = PythonOperator(task_id="check_index_counts",
                           python_callable=verify_counts)
    detect >> parse >> embed >> prune >> verify
```

CORTEX • SQL

10 lines

```
-- on the search service
target_lag = '1 hour'

-- upstream dbt models, hourly
create task refresh_docs
  warehouse = transform_wh
  schedule  =
    'USING CRON 0 * * * * Europe/London'
as
  execute dbt project neo_nexus_dbt
    args = 'build --select +doc_chunks';
```

Takeaway: the index refreshes itself; you only schedule the dbt models that feed it.

Text-to-SQL: prompt engineering becomes a semantic contract

TRADITIONAL RAG • PYTHON

15 lines

```

SCHEMA = get_ddl_for(["fct_rewards", "dim_store"]) # dump DDL
RULES = """Redeemed rewards: reward_status = 'Redeemed'.
Join rewards to stores on store_id.
'Area' means region."""

def ask_sql(question):
    prompt = (f"{SCHEMA}\n{RULES}\nWrite one Snowflake SQL "
              f"query answering: {question}\nReturn SQL only.")
    sql = generate(prompt).strip("` \n")
    if not sql.lower().startswith("select"):
        raise ValueError("unsafe SQL")
    try:
        return sql, run_readonly(sql)
    except Exception as e:
        # self-repair loop
        sql = generate(f"{prompt}\nPrevious SQL failed: {e}")
        return sql, run_readonly(sql)

```

CORTEX • PYTHON

10 lines

```

r = requests.post(ANALYST_URL,
                  headers=AUTH, json={
                      "messages": [{"role": "user",
                                      "content": [{"type": "text",
                                                      "text": question}]}],
                      "semantic_view":
                          "NEO_NEXUS.SERVING.REWARDS_SV"})

sql = next(c["statement"]
           for c in r.json()["message"]["content"]
           if c["type"] == "sql")

```

Takeaway: business rules move out of a prompt string into a governed semantic view that dbt can generate and test.

Access control: application logic becomes grants

TRADITIONAL RAG • PYTHON

15 lines

```

USER_REGIONS = load_entitlements()    # synced from IAM

def allowed_regions(user):
    regions = USER_REGIONS.get(user.id, [])
    if not regions:
        raise PermissionError("no entitlement")
    return regions

def secure_search(user, q):
    hits = search_any(q, allowed_regions(user))
    audit_log.write(user.id, q, [h.chunk_id for h in hits])
    return hits

def secure_sql(user, sql):
    sql = inject_row_filter(sql, allowed_regions(user))
    audit_log.write(user.id, sql)
    return run_readonly(sql)

# ...and again in the vector DB, the keyword engine and LLM logs

```

CORTEX • SQL

11 lines

```

create role loyalty_assistant;

grant usage on cortex search service
serving.loyalty_search
to role loyalty_assistant;

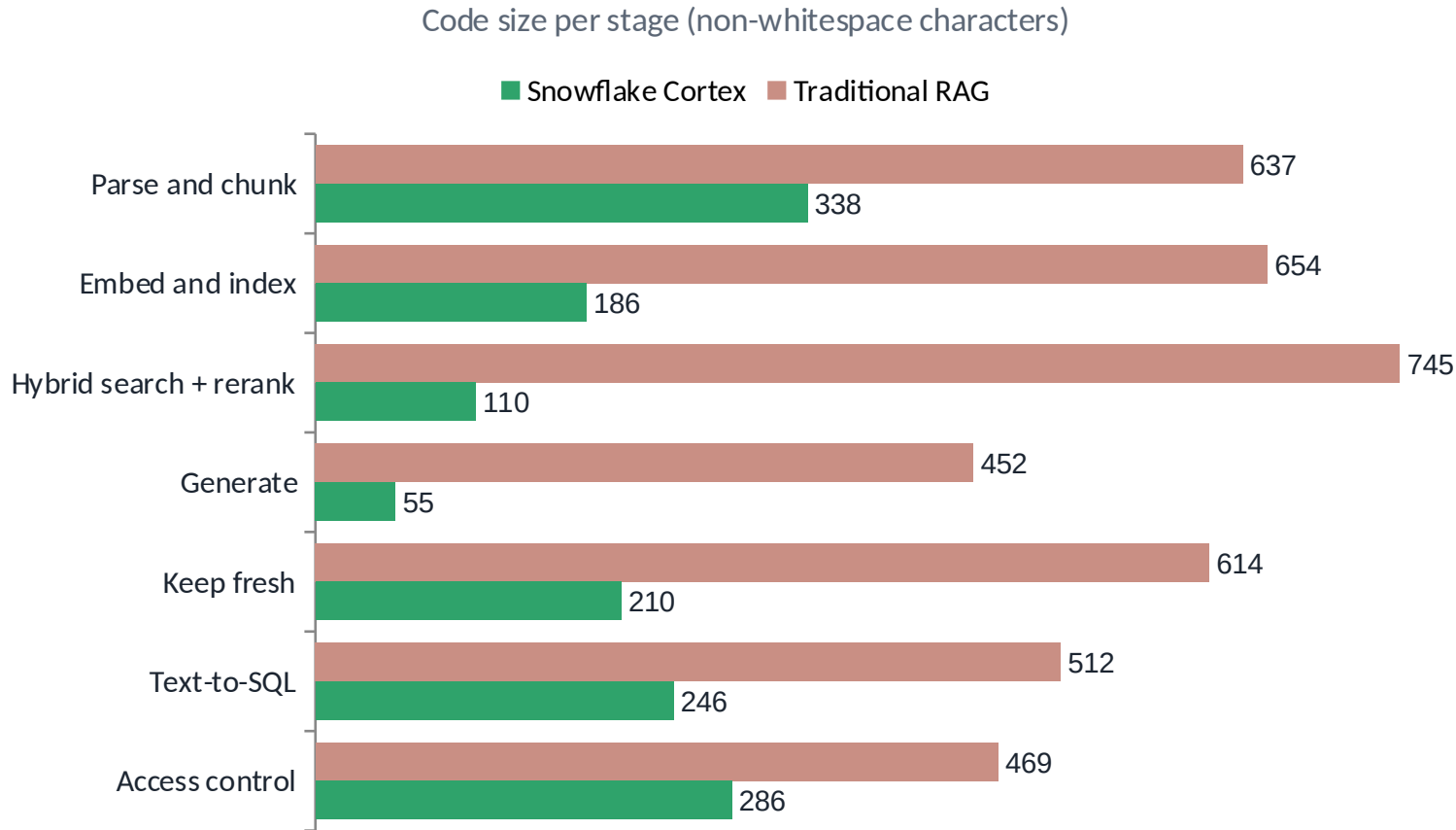
grant select on semantic view
serving.rewards_sv
to role loyalty_assistant;

-- Analyst SQL runs as the caller:
-- row access policies on the marts
-- apply automatically; queries are
-- in ACCESS_HISTORY for audit

```

Takeaway: permissions live once, in the platform, instead of being re-implemented in every system the data was copied to.

About two-thirds less code across the same seven stages

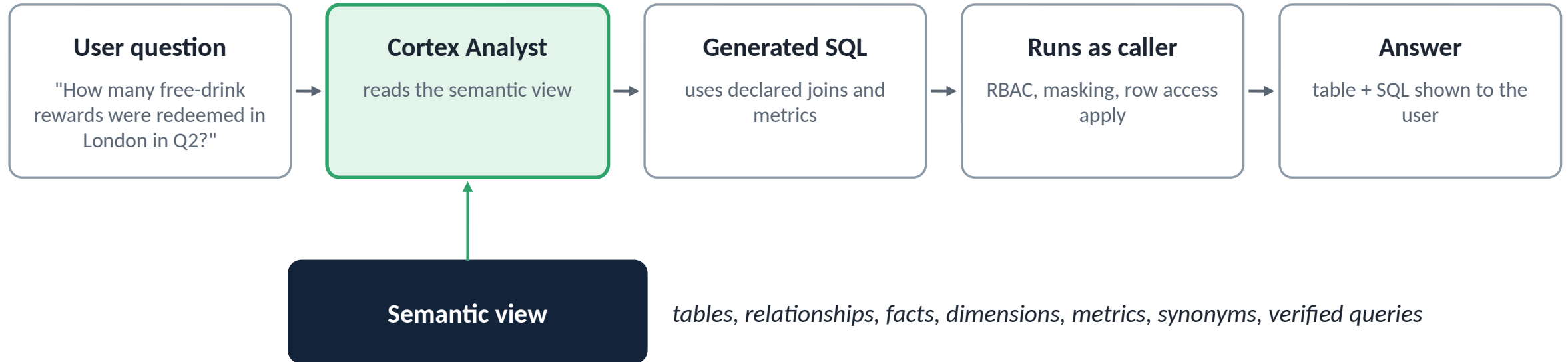


4,083 → 1,431

characters of code, about 65% less. In
lines: 115 → 67.

Not counted, traditional side: Elasticsearch,
Airflow, reranker hosting, secrets. **Not**
counted, Cortex side: the semantic view DDL.

Text-to-SQL that is grounded in business definitions



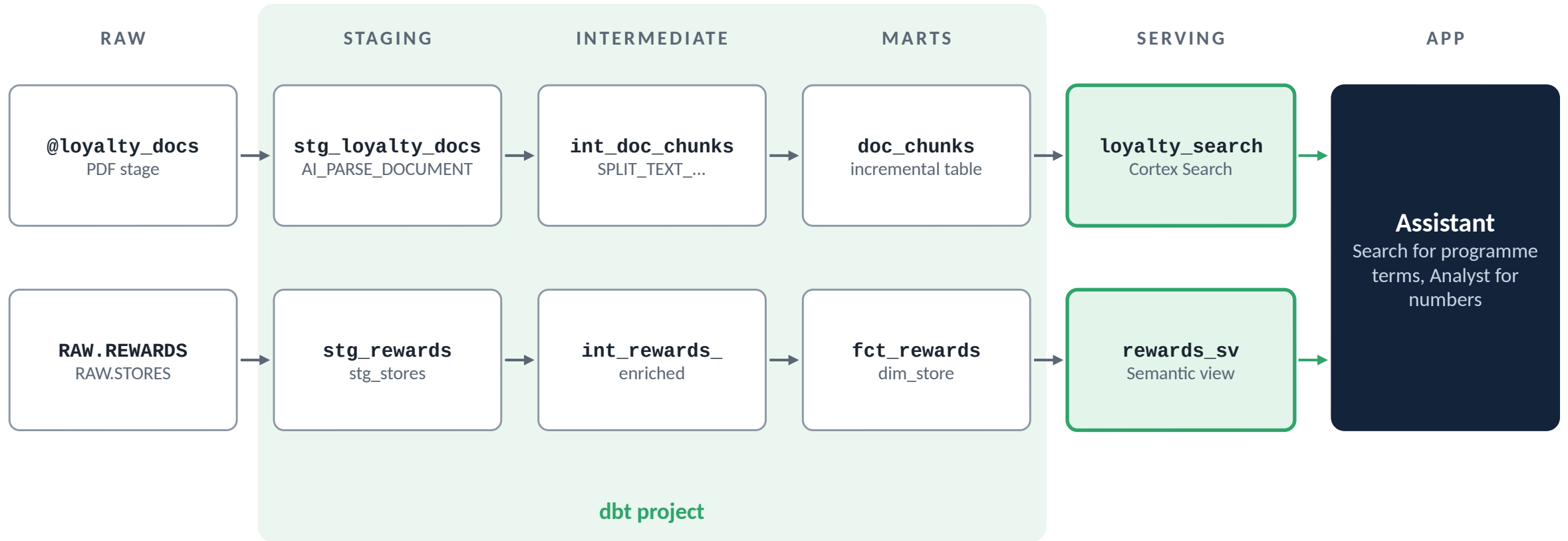
Why it beats Module 1 for numbers: it computes over every row instead of reading five similar ones, and it shows its SQL so answers are auditable.

02

dbt as the AI foundation

Staging, intermediate and mart models for both tables and documents, with Cortex services built on the marts

dbt owns the path from raw to AI-ready



Rule: the assistant reads marts only. Never RAW.

Organise the dbt project by layer and by purpose

REPOSITORY

```
neo_nexus_dbt/
├── models/
│   ├── staging/
│   │   ├── _sources.yml
│   │   ├── stg_rewards.sql
│   │   ├── stg_stores.sql
│   │   └── stg_loyalty_docs.sql
│   ├── intermediate/
│   │   ├── int_rewards_enriched.sql
│   │   └── int_doc_chunks.sql
│   └── marts/
│       ├── _marts.yml      # tests + docs
│       ├── fct_rewards.sql
│       ├── dim_store.sql
│       └── doc_chunks.sql
├── macros/
│   ├── create_search_service.sql
│   └── create_semantic_view.sql
└── dbt_project.yml
```



Staging

1:1 with sources. Rename, cast, decode codes. No joins.



Intermediate

Joins, enrichment, and document parsing into chunks.



Marts

The contract with consumers. The AI is now one of them.



Macros

Create Cortex objects, versioned beside their models.

Staging decodes; tests prove it

DBT MODEL

```
-- models/staging/stg_rewards.sql
with src as (
  select * from {{ source('raw', 'rewards_export') }}
  qualify row_number() over (
    partition by rwd_id order by _loaded_at desc) = 1
)
select
  rwd_id                as reward_id,
  store_no              as store_id,
  decode(rwd_sts, 'R', 'Redeemed', 'I', 'Issued',
    'E', 'Expired', 'C', 'Cancelled')
    as reward_status,
  try_to_number(amt, 12, 2) as reward_value,
  coalesce(try_to_date(iss_dt, 'YYYYMMDD'),
    try_to_date(iss_dt, 'DD/MM/YYYY'))
    as issued_date
from src
```

DBT TESTS + DOCS

```
# models/staging/_staging.yml
models:
  - name: stg_rewards
    description: One row per reward.
    columns:
      - name: reward_id
        tests: [unique, not_null]
      - name: reward_status
        description: Final status.
        tests:
          - accepted_values:
              values: [Redeemed, Issued,
                Expired, Cancelled]
      - name: issued_date
        tests: [not_null]
```

Parsing and chunking become SQL models

PARSE

```
-- models/staging/stg_loyalty_docs.sql
{{ config(materialized='incremental',
          unique_key='doc_id') }}

select
  relative_path                as doc_id,
  split_part(relative_path, '/', 1) as channel,
  last_modified,
  ai_parse_document(
    to_file('@raw.loyalty_docs', relative_path),
    {'mode': 'LAYOUT'})
    :content::string            as doc_text
from directory(@raw.loyalty_docs)
{% if is_incremental() %}
where last_modified >
  (select max(last_modified) from {{ this }})
{% endif %}
```

CHUNK

```
-- models/intermediate/int_doc_chunks.sql
select
  d.doc_id,
  d.channel,
  c.index                as chunk_no,
  d.doc_id || '#' || c.index as chunk_id,
  c.value::string        as chunk_text
from {{ ref('stg_loyalty_docs') }} d,
lateral flatten(
  input => snowflake.cortex
    .split_text_recursive_character(
      d.doc_text,
      'markdown', -- respect headings
      1800,       -- chunk size (chars)
      300)        -- overlap
) c
```

Create the search service from dbt, next to its source

MART

```
-- models/marts/doc_chunks.sql
{{ config(
    materialized = 'incremental',
    unique_key   = 'chunk_id',
    post_hook    = [
        "alter table {{ this }}"
        "set change_tracking = true",
        "{{ create_search_service(this) }}"
    ]
) }}
select chunk_id, doc_id, channel,
       chunk_no, chunk_text
from {{ ref('int_doc_chunks') }}
```

MACRO

```
-- macros/create_search_service.sql
{% macro create_search_service(rel) %}
create cortex search service if not exists
    {{ rel.database }}.serving.loyalty_search
on chunk_text
attributes channel, doc_id
warehouse = transform_wh
target_lag = '1 hour'
as
select chunk_id, doc_id,
       channel, chunk_text
from {{ rel }}
{% endmacro %}
```

Declare the business meaning once, on the marts

SNOWFLAKE SQL

```
create or replace semantic view serving.rewards_sv
  tables (
    rewards  as marts.fct_rewards primary key (reward_id)
             comment = 'One row per reward issued',
    stores   as marts.dim_store primary key (store_id)
  )
  relationships (
    reward_store as rewards (store_id) references stores
  )
  facts (
    rewards.reward_value as reward_value
  )
  dimensions (
    stores.region as region
    with synonyms = ('area', 'city'),
    rewards.reward_status as reward_status,
    rewards.issued_date as issued_date
  )
  metrics (
    rewards.reward_count as count(rewards.reward_id),
    rewards.total_value as sum(rewards.reward_value)
  );
```

Tables

marts only, with primary keys

Relationships

joins declared, never inferred

Synonyms

how people talk, mapped to columns

Metrics

one formula per KPI, for everyone

03

The assistant

Answering from documents and from tables, routing between them, and keeping it governed

The Module 1 answer function, rebuilt on Cortex

PYTHON • SNOWPARK

```
from snowflake.core import Root
from snowflake.cortex import complete

svc = (Root(session).databases["NEO_NEXUS"]
       .schemas["SERVING"].cortex_search_services["LOYALTY_SEARCH"])

def answer(question, channel=None):
    kw = {"filter": {"@eq": {"channel": channel}}} if channel else {}
    hits = svc.search(query=question,
                      columns=["chunk_id", "chunk_text"],
                      limit=5, **kw).results
    context = "\n\n".join(
        f"[{h['chunk_id']}] {h['chunk_text']}" for h in hits)
    prompt = (f"{SYSTEM}\n\nContext:\n{context}"
              f"\n\nQuestion: {question}")
    return complete("claude-sonnet-4-5", prompt,
                   session=session), hits
```

What changed

- No embedding model to load
- No similarity SQL to tune
- Hybrid search and reranking included
- LLM runs inside Snowflake

What stayed

The prompt, the citation contract and the abstain rule: your design, not the vendor's.

One function, many engines: change a string, not a stack

Family	Example AI_COMPLETE model strings
Anthropic Claude	claude-opus-5-5, claude-sonnet-4-5
OpenAI GPT	openai-gpt-5.5, openai-gpt-5-mini
Meta Llama	llama3.3-70b, llama3.1-8b
Mistral	mistral-large3
Google Gemini	gemini-3.1-pro

Regional availability varies; some models need cross-region inference.

Module 1: new engine = new SDK, key and infrastructure. **Cortex:** new engine = new string.

CORTEX · SQL

```
-- same prompts, three engines,  
-- one query: a model bake-off  
select  
    question_id,  
    ai_complete('claude-sonnet-4-5',  
                prompt) as claude,  
    ai_complete('openai-gpt-5-mini',  
                prompt) as gpt,  
    ai_complete('llama3.3-70b',  
                prompt) as llama  
from eval.golden_prompts;
```

Pick the engine per task, and govern which engines are allowed

1

Route: classify DOCUMENT vs ANALYTICAL

claude-haiku-4-5 · llama3.1-8b

2

Answer: grounded response with citations

claude-sonnet-4-5 · openai-gpt-5-mini

3

Judge: score answers in evaluation (Module 3)

claude-opus-5-5 · openai-gpt-5.5

GOVERNANCE • SQL + PYTHON

```
-- admin: only approved engines
alter account set cortex_models_allowlist =
    'claude-sonnet-4-5,claude-haiku-4-5,llama3.3-70b';

-- admin: allow other regions (residency call)
alter account set cortex_enabled_cross_region =
    'AWS_EU';

# Module 1's OpenAI-style client, pointed at Cortex
client = OpenAI(
    api_key=SNOWFLAKE_PAT,
    base_url=f"https://{ACCOUNT}.snowflakecomputing.com"
              "/api/v2/cortex/v1")
client.chat.completions.create(
    model="claude-sonnet-4-5", messages=msgs)
```

Two routes, one assistant

CORTEX ANALYST • REST

```
resp = requests.post(
    f"https://{ACCOUNT}.snowflakecomputing.com"
    "/api/v2/cortex/analyst/message",
    headers={"Authorization": f"Bearer {TOKEN}",
            "Content-Type": "application/json"},
    json={"messages": [{"role": "user", "content": [
        {"type": "text", "text": question}]}],
        "semantic_view": "NEO_NEXUS.SERVING.REWARDS_SV"})

content = resp.json()["message"]["content"]
sql = next(c["statement"] for c in content
           if c["type"] == "sql")
df = session.sql(sql).to_pandas()  # runs as the user
```

1

Classifier prompt

AI_COMPLETE labels the question: DOCUMENT or ANALYTICAL. Transparent and easy to test.

2

Cortex Agent

Search and Analyst registered as tools; the agent plans and calls them. Covered in Module 6.

The assistant inherits Snowflake's security model



Least-privilege role

USAGE on the search service, SELECT on marts and the semantic view. Nothing on RAW.



Policies still apply

Analyst SQL runs as the caller: row access and masking policies filter every answer.



Data stays in place

Models run inside Snowflake. Cross-region inference is an explicit admin setting.



Cost is observable

Refresh warehouse, serving and tokens are metered and visible in ACCOUNT_USAGE.

What we gained, and what we gave up

Dimension	Module 1: DIY	Module 2: Cortex + dbt
Code to maintain	Ingest, index, retrieval, LLM client	SQL models, one service, one view
Retrieval	Vector only (hybrid as stretch)	Hybrid + reranking built in
Analytical questions	Not supported	Cortex Analyst
Freshness	Re-run a script	Declared: TARGET_LAG, dbt schedule
Governance	Application code	Snowflake RBAC and policies
Control over internals	Full	Limited: managed embeddings and reranker
Portability	Any Postgres, any LLM	Snowflake-specific

Decide with evidence: the lab compares recall@5 on the same golden set.

Rebuild the Neo Nexus assistant in Snowflake

1

Set up

load raw tables; upload PDFs to @raw.loyalty_docs

10 min

2

Model tables

stg, int and marts for rewards and stores, with tests

25 min

3

Model documents

parse, chunk, doc_chunks, search service

20 min

4

Semantic view

rewards_sv; ask five analytical questions

15 min

5

Compare

rerun the Module 1 golden set; recall@5 old vs new

20 min

Deliverables

- dbt project: all models build, all tests pass
- loyalty_search service and rewards_sv semantic view
- Recall@5: Module 1 vs Module 2 on the same questions
- Half-page note: which change moved recall most, and why

Stretch: verified queries for the top three analytical questions

Questions worth arguing about

1. Where should a metric's definition live: dbt, the semantic view, or both? How do you stop them drifting?
2. Cortex Search hides its embedding and reranking choices. What do you lose, and how would you detect a regression?
3. Is there ever a good reason to let an assistant read RAW?
4. Who should own the semantic view: data engineering, analytics or the business?



References

Snowflake docs: Cortex Search overview; CREATE CORTEX SEARCH SERVICE

Snowflake docs: Semantic views; Cortex Analyst REST API

Snowflake docs: AI_PARSE_DOCUMENT;
SPLIT_TEXT_RECURSIVE_CHARACTER

dbt Labs: How we structure our dbt projects

Kimball and Ross (2013) The Data Warehouse Toolkit, 3rd ed. Wiley.

SnowPro prep: Cortex and AI topics in the current SnowPro Core study guide

Learning from conversations



Log every turn

Questions, retrieved chunks, SQL, answers and latency, stored in Snowflake



Capture feedback

Thumbs, corrections and abandonment as training signal



Evaluate and tune

Golden sets, LLM-as-judge, prompt and retrieval tuning, cost

Before Module 3: finish the lab, keep your recall@5 results, and add 10 new questions from a colleague who has never seen the data.

Evaluating and Optimising AI Assistants

Learning from conversations: logging, feedback, evaluation, and improving the assistant one measured change at a time

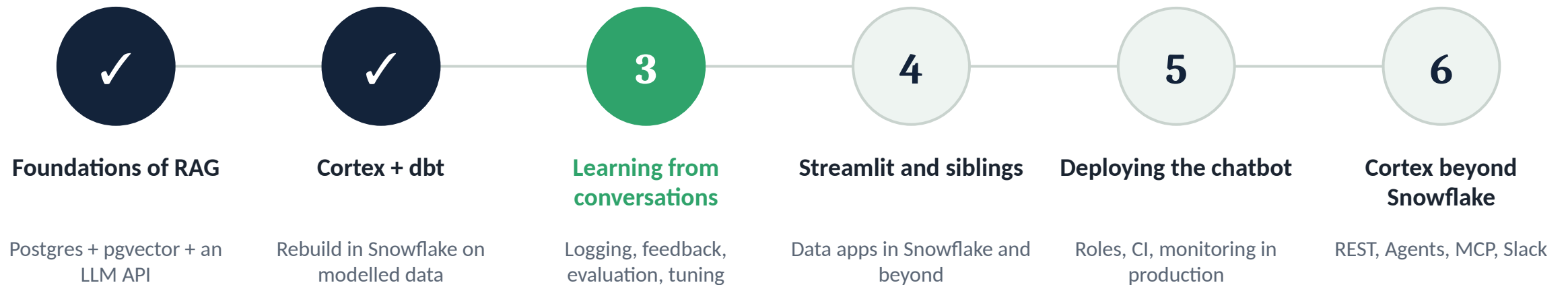
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



From a working assistant to an improving one



Today's output: conversation logs, an evaluation set, an automated judge, and one measured improvement with before and after numbers.

By the end of today you can...



Instrument

an assistant so every turn is logged with its route, retrieved context, model, prompt version, cost and feedback.



Evaluate

retrieval, answers and SQL with a curated test set and a calibrated LLM judge, all in SQL.



Diagnose

failures from real conversations and trace each one to the layer that caused it.



Optimise

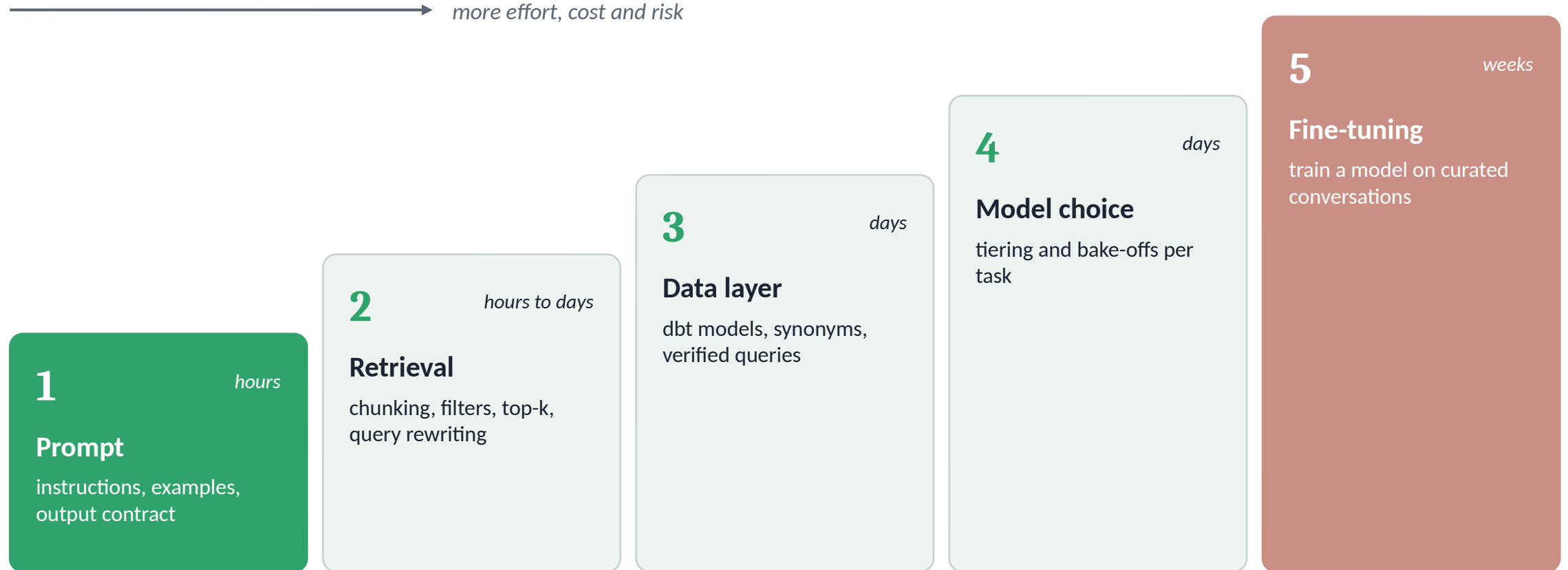
with the cheapest effective lever first, and ship only changes that pass a regression gate.

"Training the chatbot" rarely means training the model

Meaning	What changes	Who does it	Use it for
Pre-training	The base model's weights, from scratch	Model vendor	<i>Never your job</i>
Fine-tuning	Weights, on your prompt and answer pairs	You, occasionally	Style, format, distilling to a smaller model
System improvement	Data, retrieval, prompts, routing, model choice	You, continuously	Knowledge, accuracy, freshness, cost

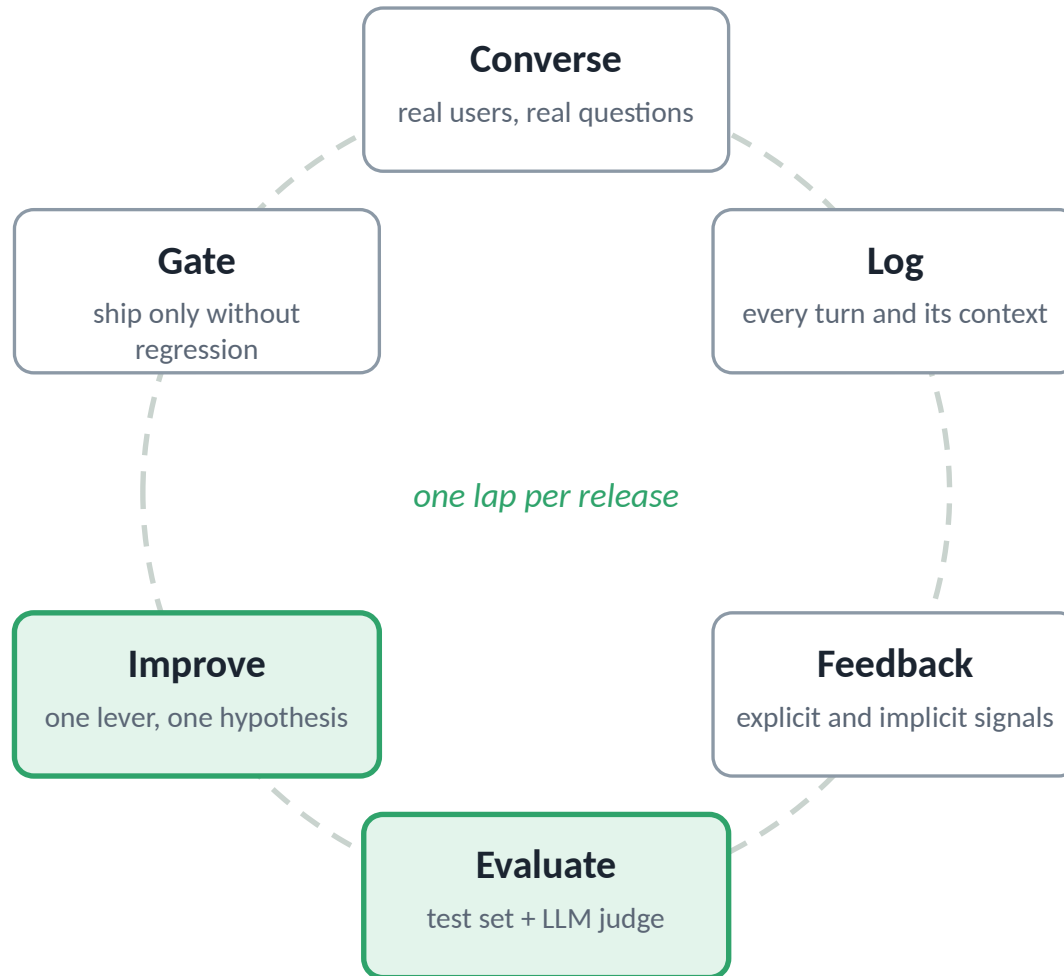
Rule of thumb: if the assistant does not know something, fix the data or retrieval. If it knows but answers badly, fix the prompt. Fine-tune only when a cheaper model must behave like an expensive one.

Climb from the cheapest lever to the most expensive



Start at rung 1. Climb only when the evidence says the lower rung cannot fix it.

Conversations become evidence; evidence becomes changes



Part 1 • Capture

Log and Feedback: what happened in each turn, and was it good?

Part 2 • Evaluate

Evaluate: a repeatable score for the current version, and a map of its failures.

Part 3 • Optimise

Improve and Gate: pull the cheapest effective lever and prove it helped.

01

Capture

Log every turn with its context, and collect the signals that tell you whether it went well

Log enough to replay and diagnose any turn

Request

question, standalone question, session, user role, timestamp

Route

DOCUMENT or ANALYTICAL, and the router's model

Retrieval

chunk IDs and scores, filters applied

Generation

model, prompt version, tokens in and out, latency

Output

answer text, generated SQL, abstained or not

Feedback

thumbs, correction text, follow-up behaviour



Logs are customer data. Apply masking policies to free text, set a retention period, and restrict who can read them.

Two tables in Snowflake: turns and feedback

SNOWFLAKE SQL

```
create table obs.chat_turns (  
  turn_id      string default uuid_string(),  
  session_id   string,  
  user_role    string,  
  asked_at     timestamp_ltz default current_timestamp(),  
  question     string,  
  standalone_q string,  
  route        string,          -- DOCUMENT | ANALYTICAL  
  chunk_ids     array,  
  chunk_scores  array,  
  generated_sql string,  
  model         string,  
  prompt_version string,  
  input_tokens  number,  
  output_tokens number,  
  latency_ms    number,  
  answer        string  
);  
  
create table obs.feedback (  
  turn_id   string,  
  rating    number,          -- +1 / -1  
  correction string,  
  created_at timestamp_ltz default current_timestamp()  
);
```

Append-only

one row per turn; never updated

Arrays

chunk IDs and scores stay with the turn

Separate feedback

arrives later, joined on turn_id

Modelled in dbt

stg_chat_turns feeds evaluation

One function wraps every turn

PYTHON • SNOWPARK

```
def chat(session_id, history, message):
    t0 = time.perf_counter()
    q = standalone(history, message)          # Part 3: multi-turn
    route = classify(q)                       # DOCUMENT | ANALYTICAL
    if route == "DOCUMENT":
        answer, hits = answer_docs(q); sql = None
    else:
        answer, sql = answer_analytical(q); hits = []

    session.sql("""
        insert into obs.chat_turns
        (session_id, user_role, question, standalone_q, route,
         chunk_ids, generated_sql, model, prompt_version,
         latency_ms, answer)
        select ?, current_role(), ?, ?, ?, parse_json(?),
               ?, ?, ?, ?, ?""",
        params=[session_id, message, q, route,
                json.dumps([h["chunk_id"] for h in hits]), sql,
                MODEL, PROMPT_VERSION,
                int((time.perf_counter() - t0) * 1000), answer]
    ).collect()
    return answer
```

Build it yourself

Full control of what is captured; the tables join directly to marts and eval runs.

Or use the managed option

Snowflake AI Observability traces Cortex apps; TruLens covers apps hosted elsewhere.

Most feedback is implicit: read behaviour, not just thumbs

Signal	Type	Usually means	Strength
Thumbs down	Explicit	the answer was wrong or unhelpful	Strong, but rare
Typed correction	Explicit	the right answer, in the user's words	Gold
Rephrase within a minute	Implicit	the first answer missed	Strong
Session ends after an abstain	Implicit	a coverage gap in the data	Medium
Escalation to a human	Implicit	out of scope or low confidence	Medium
Answer copied	Implicit	the answer was used	Weak positive

Treat implicit signals as review candidates, not ground truth.

02

Evaluate

A curated test set, metrics for every layer, and an LLM judge you have checked against people

Build the test set from four sources, not one



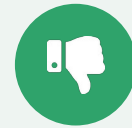
Golden set

Module 1's hand-written questions with expected chunks



Fresh eyes

the colleague questions from Module 2's homework



Mined from logs

thumbs-down and rephrased turns, reviewed and labelled



Adversarial

out of scope, prompt injection, requests for personal data

Stratify by route (document or analytical), channel, and difficulty, so regressions show up per slice.

Every fixed production failure becomes a permanent test case.

Measure every layer, so a failure points to its cause

Layer	Metric	How it is computed
Retrieval	Recall@5, MRR	expected chunk IDs vs retrieved chunk IDs
Answer	Faithfulness, relevance	LLM judge with a rubric, calibrated on human labels
Analytical SQL	Execution accuracy	generated SQL returns the same rows as the reference SQL
Behaviour	Correct abstain, correct refusal	adversarial slice, judged or rule-checked
Operations	p95 latency, cost per answer	from obs.chat_turns and Cortex usage views

Always report per slice. An average can improve while one channel gets worse.

Grade thousands of answers in one SQL statement

SNOWFLAKE SQL

```
create or replace table eval.judged as
select r.run_id, r.question_id,
       try_parse_json(ai_complete('claude-opus-5-5',
       'You grade a loyalty assistant. Use ONLY the ' ||
       'context. Return JSON: {"reasoning": string, ' ||
       '"faithfulness": 1-5, "relevance": 1-5, ' ||
       '"should_abstain": boolean}' ||
       '\n\nContext: ' || r.context ||
       '\n\nQuestion: ' || r.question ||
       '\n\nAnswer: ' || r.answer)) as verdict
from eval.run_results r;

select run_id,
       avg(verdict:faithfulness::number) as faithfulness,
       avg(verdict:relevance::number)   as relevance
from eval.judged
group by run_id;
```

Reason, then score

consistency improves when the judge explains first

Different model family

judges tend to favour their own style

Anchored 1 to 5 scale

describe what a 1, 3 and 5 look like

Calibrate on 30 human labels

trust the judge only once it agrees with people

Judge SQL by its results, not its text

SNOWFLAKE SQL

```
-- 1 when both queries return exactly the same rows
select iff(count(*) = 0, 1, 0) as exact_match
from (
  ( select * from table(result_scan(:generated_qid))
    minus
    select * from table(result_scan(:reference_qid)) )
  union all
  ( select * from table(result_scan(:reference_qid))
    minus
    select * from table(result_scan(:generated_qid)) )
);
```

Why not compare SQL text?

Different joins, aliases and CTEs can be equally correct.
Only the results matter to the user.

Normalise first

Align column order and names; round totals before comparing.

Store the reference SQL for every analytical test case. It doubles as a verified query candidate in Part 3.

Classify what went wrong, then fix the right layer

SNOWFLAKE SQL

```
select
  ai_classify(
    'Question: ' || t.question ||
    ' Answer: ' || t.answer ||
    ' Correction: ' || coalesce(f.correction, ''),
    ['retrieval miss', 'wrong SQL',
    'unsupported claim', 'out of scope',
    'formatting']
  ):labels[0]::string as failure_type,
  count(*) as turns
from obs.chat_turns t
join obs.feedback f using (turn_id)
where f.rating = -1
group by failure_type
order by turns desc;
```

Failure type	Fix at rung
Retrieval miss	2 • Retrieval
Wrong SQL	3 • Semantic layer
Unsupported claim	1 • Prompt
Out of scope	1 • Prompt / routing
Formatting	1 • Prompt, or 5 • Fine-tune

Read ten turns per class before acting: the classifier triages, people decide.

03

Optimise

Climb the ladder: prompt, retrieval, semantic layer, model choice, and fine-tuning only when earned

Version prompts like code, and test them like code

SNOWFLAKE SQL

```
create table app.prompts (
  prompt_version string,      -- answer_v7
  purpose        string,      -- answer | route
  template       string,
  is_active      boolean,
  created_by     string,
  created_at     timestamp_ltz
                default current_timestamp()
);

-- the version used is logged on every turn:
-- obs.chat_turns.prompt_version
```

PYTHON • A/B SPLIT

```
def pick_prompt(session_id):
    h = hashlib.sha1(session_id.encode())
    bucket = int(h.hexdigest(), 16) % 100
    return ("answer_v8" if bucket < 10
            else "answer_v7")    # 10% canary
```

Offline before online

Run both versions on the evaluation set; only A/B test the winner with users.

Conversations need standalone questions before retrieval

PYTHON • QUERY REWRITING

```
REWRITE = ("Rewrite the user's last message as a "
           "standalone question using the conversation. "
           "Return only the question.")

def standalone(history, message):
    convo = "\n".join(f"{t['role']}: {t['text']}"
                      for t in history[-6:])
    return complete("claude-haiku-4-5",
                    f"{REWRITE}\n\n{convo}\nuser: {message}",
                    session=session)
```

"And in the app?" → "Can members redeem a free drink in the app?"

Then tune one knob at a time

Chunk size and overlap

dbt model parameters

Attribute filters

channel, document version

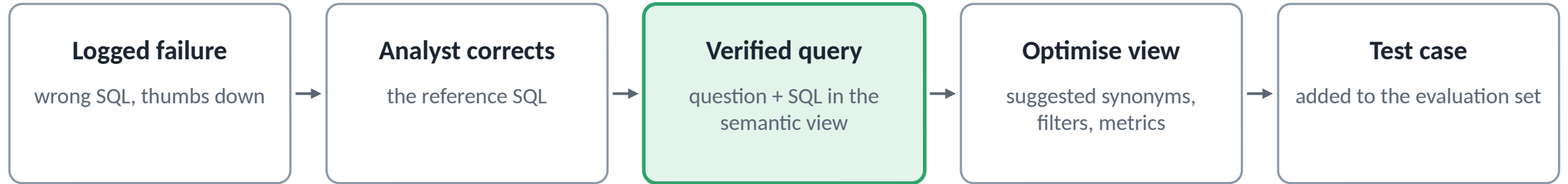
Top-k

fewer, better chunks

Source coverage

abstains reveal missing documents

Failed analytical questions become verified queries



Analytical route

Corrected SQL becomes a verified query. Snowflake can suggest candidate questions from query history and optimise the semantic view from verified queries.

Document route

Repeated abstains on a topic usually mean a missing or stale document: a dbt and ingestion fix, not a prompt fix.

Choose the cheapest model that passes the evaluation

Step	Candidates to compare	Decide on
Route	claude-haiku-4-5, llama3.1-8b	routing accuracy, latency
Answer	claude-sonnet-4-5, openai-gpt-5-mini, llama3.3-70b	faithfulness, cost per answer
Judge	claude-opus-5-5, openai-gpt-5.5	agreement with human labels

Model strings from Snowflake's list at the time of writing; availability varies by region.

Decision rule

1. Set a quality bar per step.
2. Drop models below the bar.
3. Of the rest, pick the cheapest that meets the latency target.

Fine-tune to make a small model behave like a large one

SNOWFLAKE SQL

```
-- training pairs: highly rated answers from the large model
create or replace table ft.train as
select t.standalone_q || '\n\nContext:\n' || c.context
      as prompt,
      t.answer as completion
from obs.chat_turns t
join obs.feedback f      using (turn_id)
join ft.turn_context c    using (turn_id)
where f.rating = 1 and t.route = 'DOCUMENT';

select snowflake.cortex.finetune('CREATE',
  'neonexus_answer_8b',          -- new model
  'llama3.1-8b',                 -- base model
  'select prompt, completion from ft.train',
  'select prompt, completion from ft.validation');

-- then: ai_complete('neonexus_answer_8b', prompt)
```

Good reasons

Cut cost or latency on a narrow task; enforce a consistent format or tone.

Bad reasons

Teaching the model new facts (use retrieval); fixing a problem you have not diagnosed.

Optimise cost and speed without touching quality



Tier the models

small for routing and rewriting, mid for answers, large only for judging



Cap the context

top-k of 3 to 5 good chunks beats 20 average ones



Cache repeat answers

key on standalone question + index version

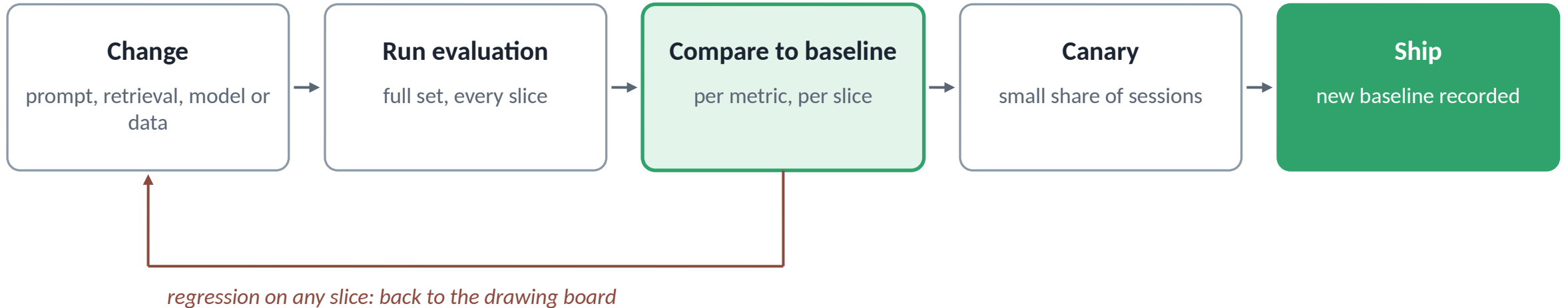


Stream responses

same total time, far better perceived speed

Monitor: CORTEX_AI_FUNCTIONS_USAGE_HISTORY and CORTEX_SEARCH_SERVING_USAGE_HISTORY, joined to obs.chat_turns.

No change ships without passing the regression gate



Example gate policy: no slice drops more than 2 points on faithfulness or recall@5, adversarial pass rate stays at 100%, and cost per answer does not rise more than 10%.

Close the loop on the Neo Nexus assistant

- | | | |
|---|--|--------|
| 1 | Instrument
create obs tables; add logging and feedback to the app | 15 min |
| 2 | Converse
role-play 30 turns as CRM analysts; rate each answer | 15 min |
| 3 | Evaluate
build the eval set; run the judge; check it on 10 answers | 20 min |
| 4 | Diagnose
classify thumbs-down turns; read samples per class | 15 min |
| 5 | Improve
pull one lever; rerun; pass or fail the gate | 25 min |

Deliverables

- obs.chat_turns with at least 30 logged turns
- Judge agreement with your own labels on 10 answers
- Failure breakdown by type
- One change: rung, hypothesis, before and after scores, gate decision

Stretch: add standalone-question rewriting and measure its effect on follow-up questions

Questions worth arguing about

1. Who judges the judge? How much human labelling is enough to trust an LLM grader?
2. When the evaluation set becomes the target, how do you stop overfitting to it?
3. How long should conversation logs be kept, and who may read them?
4. Which single metric would you put on the team's dashboard, and why only one?



References

Zheng et al. (2023) Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS.

Es et al. (2023) RAGAS: Automated Evaluation of RAG.

Shankar et al. (2024) Who Validates the Validators? UIST.

Huyen (2025) AI Engineering. O'Reilly.

Snowflake docs: AI Observability; Cortex Fine-tuning

Snowflake docs: Cortex Analyst verified queries and optimisation; AI cost management

Streamlit and its siblings



Streamlit basics

The script-reruns-on-every-interaction model, widgets and state



Streamlit in Snowflake

Apps that run next to the data, under Snowflake roles



The family

Gradio, Dash, Shiny, Databricks Apps: when to pick which

Before Module 4: keep your obs tables and evaluation set; next time the thumbs and corrections get a proper user interface.

Data Apps with Streamlit and Its Siblings

From a Python script to a shared, governed app: Streamlit fundamentals, Streamlit in Snowflake, and where Gradio, Dash, Shiny and Databricks Apps fit

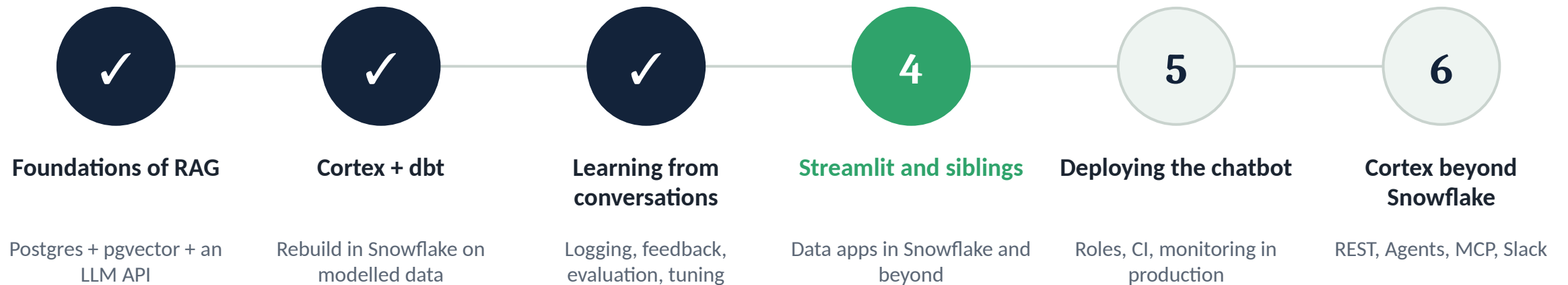


R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.

The assistant gets a user interface



Today's output: a Neo Nexus rewards explorer and a chat shell with feedback, running in Streamlit in Snowflake and shared by role.

By the end of today you can...



Explain

Streamlit's rerun execution model, and use session state and caching correctly because of it.



Build

data apps and a chat interface with feedback, using widgets, layout and chat elements.



Run

apps inside Snowflake, choose a runtime, and reason about owner's rights and sharing by role.



Choose

between Streamlit, Gradio, Dash, Shiny and hosting platforms such as Databricks Apps, with reasons.

The last mile between a dashboard and a product

BI dashboard

Power BI, Tableau, Looker

Read-only views of modelled data; great for monitoring, poor at input, logic and conversation.

Data app

Streamlit, Gradio, Dash, Shiny

Python-only, interactive, can call models and write back. Built by data teams in days.

Custom web app

React + an API, mobile apps

Full control and scale for customer-facing products; needs a front-end team and months.

faster, simpler

more control, more effort

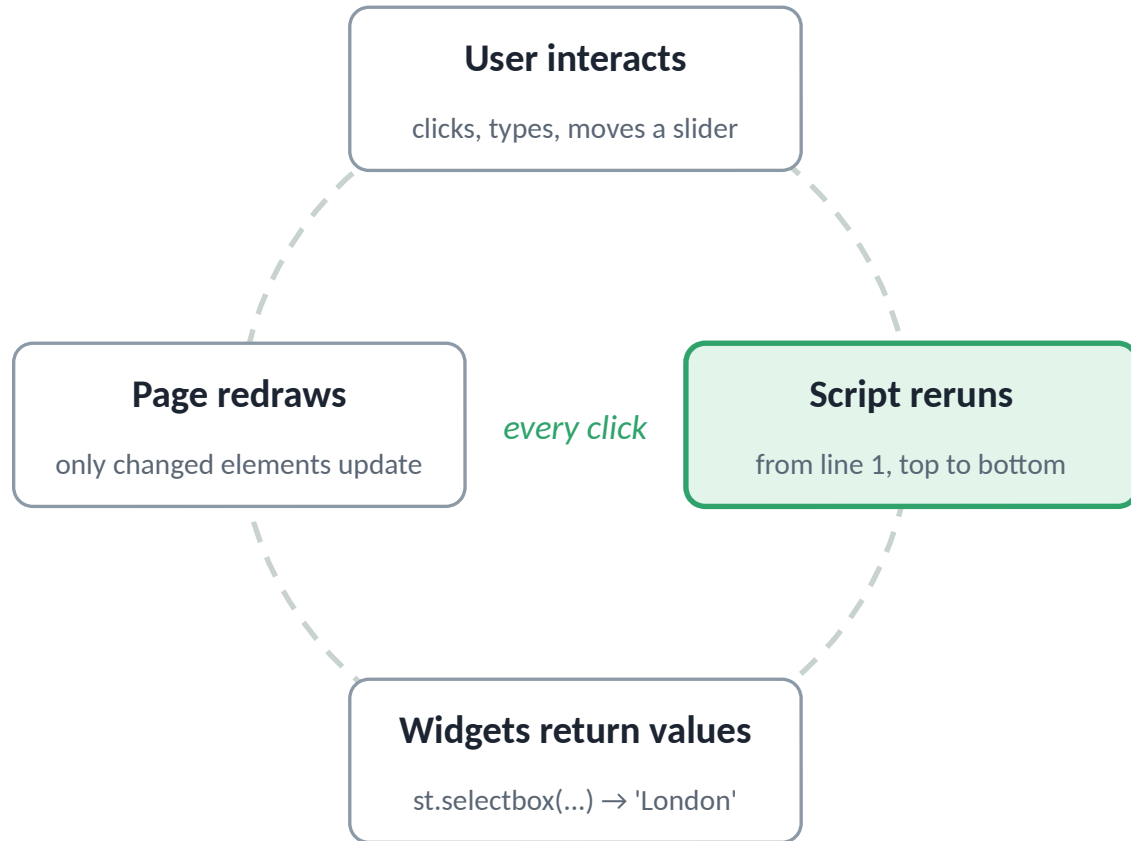
Our assistant is a data app: internal users, Python team, data in the warehouse.

01

Streamlit fundamentals

The rerun model, widgets and layout, state and caching, and the chat pattern

Every interaction reruns the whole script, top to bottom



Consequence 1 • State

Variables reset on every rerun. Anything that must persist, such as chat history, lives in st.session_state.

Consequence 2 • Caching

Expensive work runs on every click unless cached with st.cache_data or st.cache_resource.

The payoff

Code reads like a plain script: no callbacks, no front-end, no HTML.

A working data app in about fifteen lines

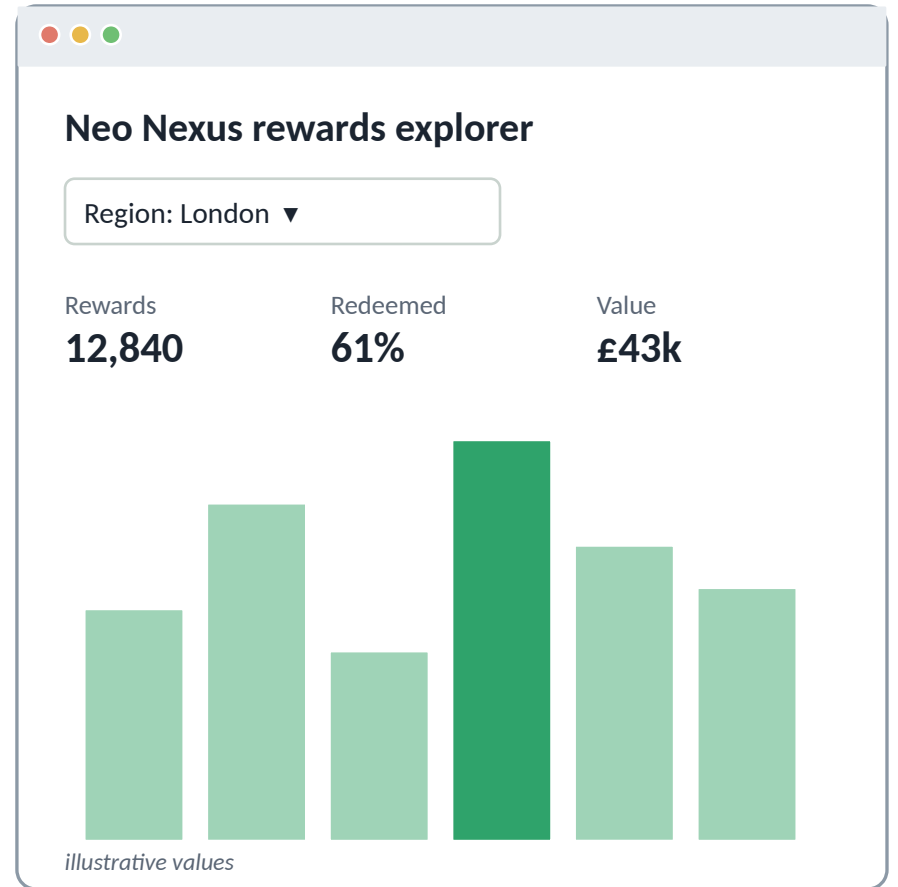
APP.PY

```
import streamlit as st

st.title("Neo Nexus rewards explorer")
region = st.selectbox("Region", ["London", "Manchester", "Edinburgh"])

@st.cache_data(ttl=600)                                # rerun-safe: cached
def load_rewards(region):
    return session.sql(
        "select * from marts.fct_rewards f "
        "join marts.dim_store p using (store_id) "
        "where p.region = ?", params=[region]
    ).to_pandas()

df = load_rewards(region)
c1, c2 = st.columns(2)
c1.metric("Rewards", len(df))
c2.metric("Redeemed", f"{{(df.REWARD_STATUS == 'Redeemed').mean():.1%}}")
st.bar_chart(df.groupby("ISSUED_MONTH").size())
```



Four families of elements cover most apps



Input

text_input, selectbox, slider, date_input, file_uploader, form



Display

dataframe, metric, bar_chart, line_chart, map, markdown



Layout

columns, tabs, sidebar, expander, container, dialog



Chat and feedback

chat_message, chat_input, write_stream, feedback

Tip: wrap related inputs in `st.form` so the script reruns once on submit, not on every keystroke.

Remember with session state; avoid repeat work with caching

PYTHON

```
# per-user memory that survives reruns
if "messages" not in st.session_state:
    st.session_state.messages = []

# results of a query: copied, refreshed after ttl
@st.cache_data(ttl=600)
def rewards_by_month(region):
    return session.sql(...).to_pandas()

# one shared object for all users: not copied
@st.cache_resource
def search_service():
    return (Root(session).databases["NEO_NEXUS"]
            .schemas["SERVING"]
            .cortex_search_services["LOYALTY_SEARCH"])
```

Use	For	Example
st.session_state	per-user values across reruns	chat history, selected filters
st.cache_data	serialisable results, with ttl	query results, API responses
st.cache_resource	shared, unserialisable objects	sessions, service handles, models

Classic mistakes: caching a connection with `cache_data`, or data with `cache_resource`.

A chat interface with feedback in about twenty lines

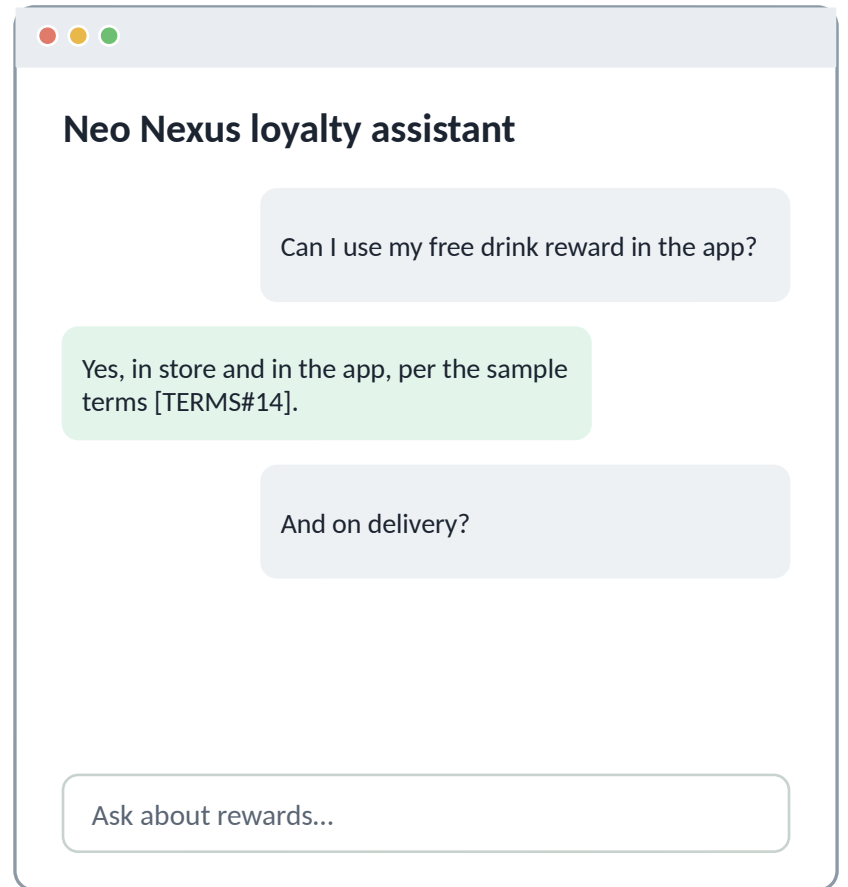
CHAT.PY

```
if "messages" not in st.session_state:
    st.session_state.messages = []

for m in st.session_state.messages:          # replay history
    st.chat_message(m["role"]).write(m["text"])

if q := st.chat_input("Ask about rewards"):
    st.chat_message("user").write(q)
    text, turn_id = answer(q, st.session_state.messages)
    st.chat_message("assistant").write(text)
    st.session_state.messages += [
        {"role": "user", "text": q},
        {"role": "assistant", "text": text, "turn_id": turn_id}]

last = st.session_state.messages[-1:] or [{}]
if tid := last[0].get("turn_id"):
    score = st.feedback("thumbs", key=f"fb_{tid}")
    if score is not None:                      # 1 = up, 0 = down
        save_feedback(tid, 1 if score == 1 else -1)
```

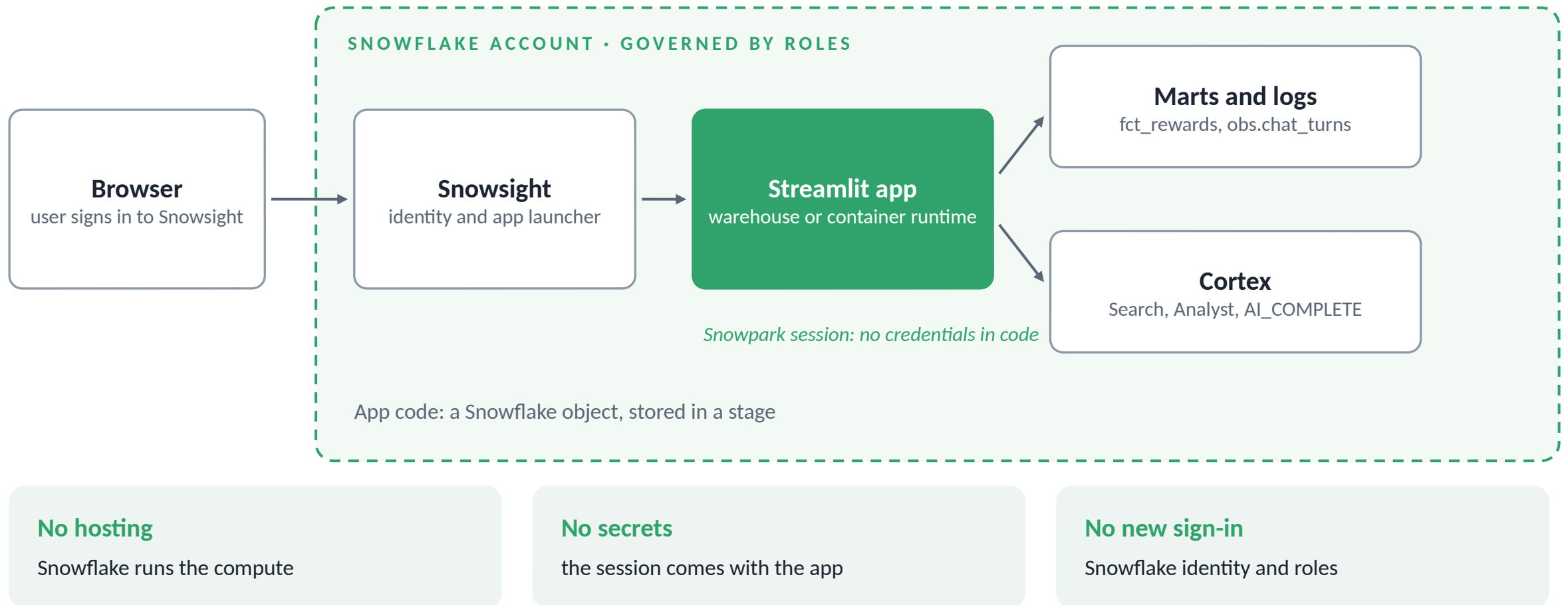


02

Streamlit in Snowflake

The same framework, running next to the data: sessions, runtimes, deployment and owner's rights

The app runs inside the platform that holds the data



Develop locally, run in Snowflake, change nothing

PYTHON • PORTABLE SESSION

```
@st.cache_resource
def get_session():
    try:
        # inside Snowflake
        from snowflake.snowpark.context import get_active_session
        return get_active_session()
    except Exception:
        # local development
        return st.connection("snowflake").session()

session = get_session()
```

LOCAL CONFIG

```
# .streamlit/secrets.toml (local only, never committed)
[connections.snowflake]
account = "..." user = "..." authenticator = "externalbrowser"
```

In Snowflake

`get_active_session()` returns the app's session. No credentials exist in the app at all.

On a laptop

`st.connection` reads `secrets.toml`. Use SSO or key-pair auth, and keep the file out of Git.

Warehouse runtime or container runtime

	Warehouse runtime	Container runtime
Compute	A virtual warehouse	A compute pool (Snowpark Container Services)
Packages	Snowflake's curated Anaconda channel	PyPI, via an external access integration
Set with	QUERY_WAREHOUSE	RUNTIME_NAME + COMPUTE_POOL
Good for	Simple apps, standard packages	Newer or niche packages, heavier apps

Either way: the SQL your app runs uses QUERY_WAREHOUSE, so size that warehouse for the queries, not the UI.

An app is a database object: create it, grant it

SNOWFLAKE SQL

```
-- upload app files to a stage, then:
create or replace streamlit apps.neo_nexus_explorer
  from @apps.code_stage/explorer
  main_file      = 'app.py'
  query_warehouse = app_wh
  title          = 'Neo Nexus rewards explorer';

-- container runtime variant adds:
--   runtime_name = 'SYSTEM$ST_CONTAINER_RUNTIME_PY3_11'
--   compute_pool = app_pool

-- share with CRM analysts
grant usage on streamlit apps.neo_nexus_explorer
  to role crm_analyst;
```

Code in a stage

Versioned alongside the rest of the project.

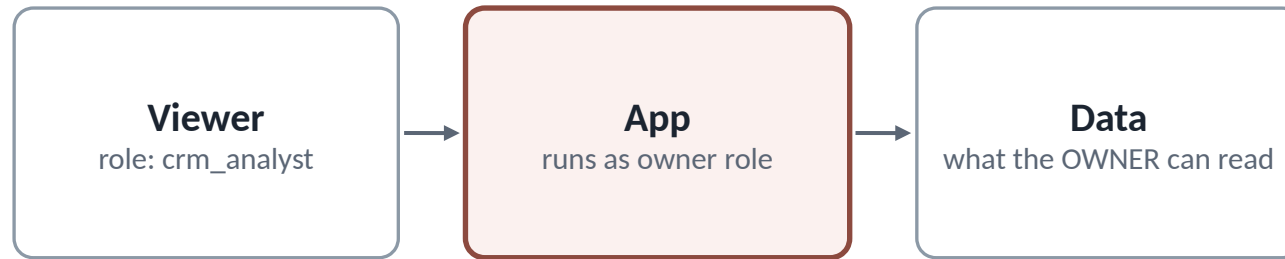
Shared by role

USAGE on the app; no public URL.

Automated in Module 5

Snowflake CLI and CI deploy the same objects.

Apps run with the owner's rights: design for it



The viewer's own grants do not limit what the app's queries can read.

So design for it

Give each app a dedicated owner role with least privilege: marts and the search service only. Keep sensitive columns masked for that role.

Different users, different rows?

Filter explicitly on the viewer's identity inside the app, and test it with two real users.

Audit

Queries appear in query history under the app's owner; log the viewer yourself in obs.chat_turns.

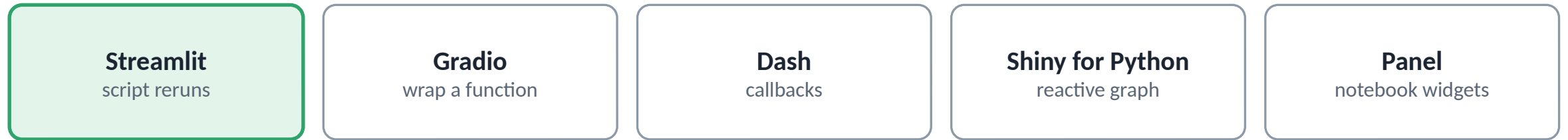
03

The family

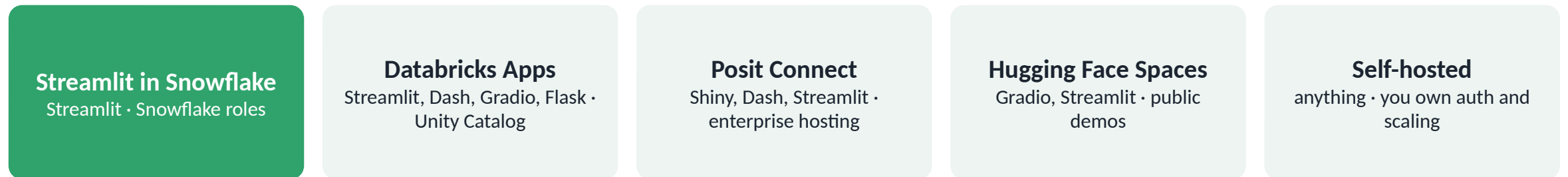
Gradio, Dash, Shiny, Panel and the platforms that host them: same problem, different trade-offs

Frameworks and platforms are separate choices

FRAMEWORKS • how you write it



PLATFORMS • where it runs and who can open it



Decide twice: the framework fits your team and interaction style; the platform fits your data, identity and governance.

Same problem, five execution models

Framework	Execution model	Strongest at	Watch out for
Streamlit	Whole script reruns on each interaction	Fastest path from script to data app; chat elements	Rerun cost in large apps
Gradio	Wraps functions; event handlers on components	Model demos, chat UIs, Hugging Face Spaces	Less suited to dashboards
Dash	Explicit callbacks, inputs to outputs	Complex, multi-page analytical apps	More boilerplate
Shiny for Python	Reactive graph: only dependants recompute	Efficient analytics; R teams moving to Python	Smaller Python ecosystem
Panel	Reactive widgets, notebook-first	Apps grown from Jupyter notebooks	Steeper styling curve

All five are open source Python frameworks; all can build a chat assistant.

The same assistant front end in three frameworks

STREAMLIT • 6 LINES

```
import streamlit as st

if q := st.chat_input("Ask"):
    st.chat_message("user") \
        .write(q)
    st.chat_message("assistant") \
        .write(answer(q))
```

GRADIO • 4 LINES

```
import gradio as gr

def respond(message, history):
    return answer(message)

gr.ChatInterface(respond).launch()
```

DASH • 14 LINES

```
from dash import (Dash, html, dcc,
                  Input, Output, State)

app = Dash()
app.layout = html.Div([
    dcc.Input(id="q"),
    html.Button("Ask", id="ask"),
    html.Div(id="a")]

@app.callback(Output("a", "children"),
              Input("ask", "n_clicks"),
              State("q", "value"),
              prevent_initial_call=True)
def respond(_, q):
    return answer(q)

app.run()
```

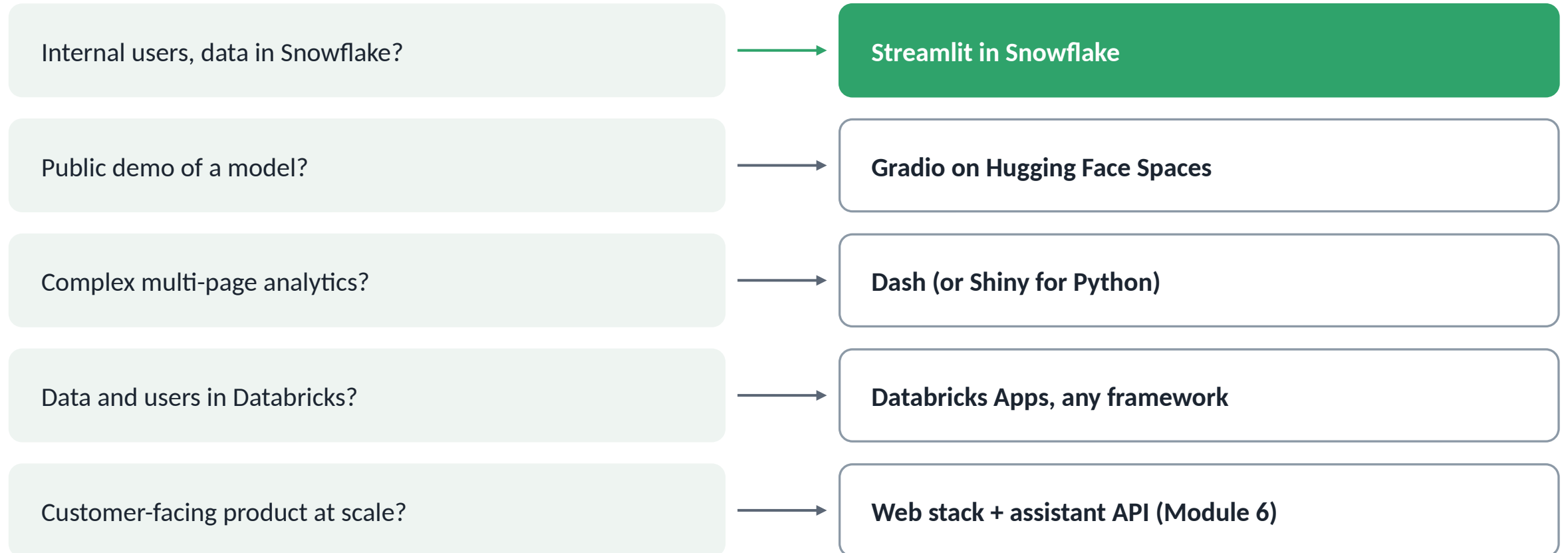
Same brain, different faces: keep `answer()` separate from the UI and you can switch frameworks in an afternoon.

Streamlit in Snowflake and Databricks Apps are close cousins

	Streamlit in Snowflake	Databricks Apps	Self-hosted
Frameworks	Streamlit	Streamlit, Dash, Gradio, Flask and more	Any
Identity	Snowflake users and roles	Databricks workspace identity	You build it (SSO, OAuth)
Data access	Snowpark session, no secrets	Unity Catalog, SQL warehouses	Credentials you manage
AI services	Cortex in the same account	Model Serving, Mosaic AI	Any API you call
Best when	Data and users live in Snowflake	Data and users live in Databricks	Public or customer-facing apps

Rule: host the app where the governed data already lives.

A short decision guide



The rest of this course: Streamlit in Snowflake.

Where Streamlit and Streamlit in Snowflake struggle



Reruns at scale

Large apps need caching, forms and fragments to stay fast.



Audience size

Built for internal users, not public, high-traffic sites.



Packages

Warehouse runtime uses the curated channel; PyPI needs the container runtime.



Outbound calls

External APIs need an external access integration.

Build the Neo Nexus explorer and chat shell

1

Local first

portable get_session(); run the rewards explorer on a laptop

15 min

2

Chat shell

chat_message, chat_input, session state; call answer()

20 min

3

Feedback

st.feedback writes to obs.feedback from Module 3

15 min

4

Deploy to Snowflake

stage the files; create streamlit; grant to a role

20 min

5

Owner's rights test

open as a narrower role; document what it can see

20 min

Deliverables

- Rewards explorer running in Streamlit in Snowflake
- Chat shell with history and thumbs feeding obs.feedback
- Shared with one role by grant, no URL sharing
- A note on what a narrower role could see, and why

Stretch: the same chat front end in Gradio, calling the same answer()

Questions worth arguing about

1. Why does Streamlit need both session state and caching, when other frameworks need neither in the same way?
2. An app runs with its owner's rights. How would you convince an auditor it is safe?
3. At what point should a data app be rebuilt as a proper web application?
4. How do you avoid being locked into one framework or one hosting platform?



References

Streamlit docs: Execution model; caching; session state; chat elements

Snowflake docs: About Streamlit in Snowflake; runtime environments

Snowflake docs: CREATE STREAMLIT; owner's rights and privileges

Databricks docs: Databricks Apps

Framework docs: Gradio ChatInterface; Dash callbacks; Shiny for Python; Panel

SnowPro prep: Streamlit in Snowflake topics in the current study guides

Deploying the chatbot



Structure the app

multi-page layout, config, secrets and environments



Ship with CI

Snowflake CLI, the Module 3 evaluation gate on every change



Run it in production

roles, monitoring, cost controls and rollback

Before Module 5: put your app code in a Git repository and install the Snowflake CLI.

Deploying AI Assistants to Production

Shipping the Neo Nexus assistant on Streamlit in Snowflake: structure, environments, CI/CD with an evaluation gate, least-privilege security and day-two operations

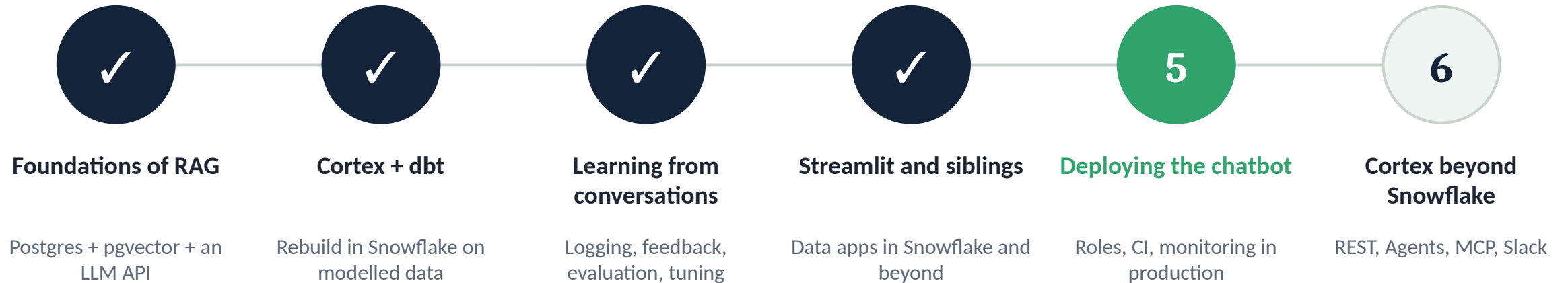
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



From a working app to a production service



Today's output: a Git repository, a CI pipeline with an evaluation gate, production roles, a health view with alerts, and a rehearsed rollback.

By the end of today you can...



Structure

the assistant as a multi-page app with thin UI, testable logic and environment-driven configuration.



Automate

deployment with the Snowflake CLI or Git integration, and block regressions with an evaluation gate in CI.



Secure

the service with least-privilege roles for the app, its users, its admins and the pipeline.



Operate

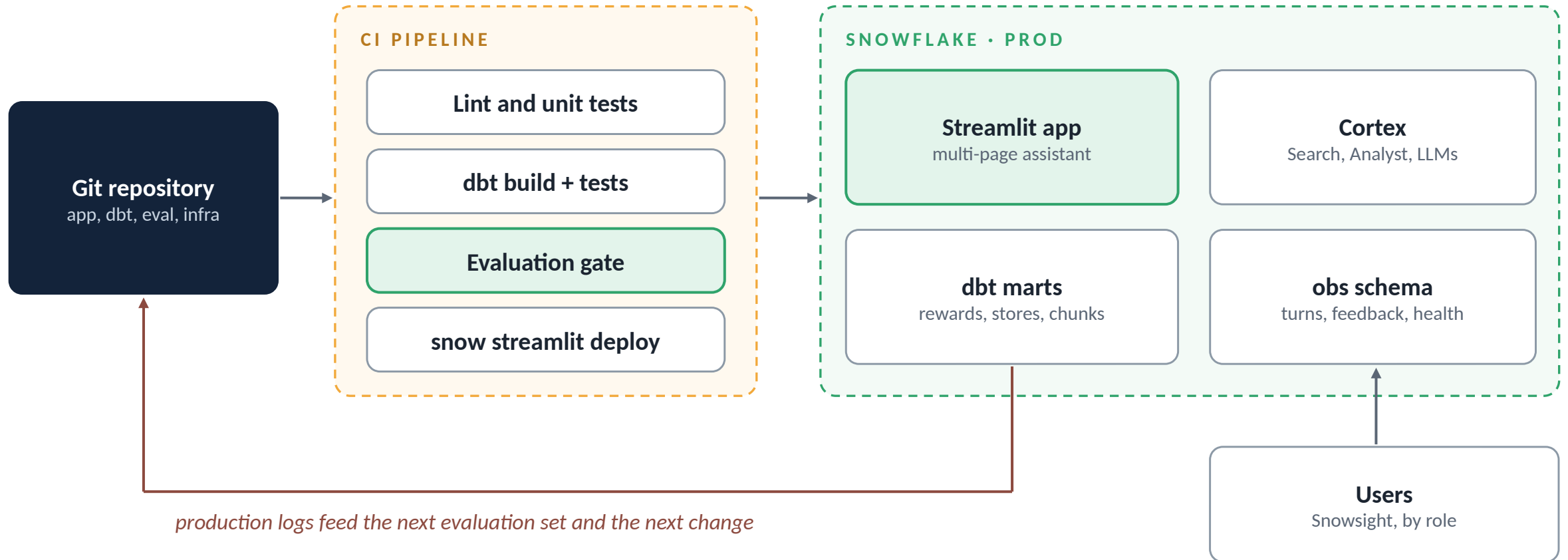
it with a health view, alerts, cost controls, a runbook and a tested rollback.

What changes between a demo and a service

Concern	Demo (Module 4)	Production (today)
Code	Files uploaded from a laptop	Git repository, reviewed pull requests
Deployment	CREATE STREAMLIT by hand	CI pipeline, identical every time
Environments	One schema	DEV, TEST and PROD from one codebase
Quality	It looked right in the demo	Evaluation gate blocks regressions
Access	Owner's role, broad grants	Least-privilege roles per purpose
Operations	Nobody is watching	Health view, alerts, runbook, rollback
Cost	Unbounded	Resource monitors and budgets

None of these rows is about the model. The production gap is engineering and operations.

Git to pipeline to Snowflake, with a loop back from production



Every box here is covered today.

01

Structure the app

A repository and app layout built for change: thin UI, testable logic, configuration by environment, graceful failure

One repository holds everything the assistant needs

REPOSITORY

```
neo-nexus-assistant/  
├── app/  
│   ├── streamlit_app.py      # entry point  
│   └── pages/  
│       ├── 1_Assistant.py  
│       ├── 2_Rewards_explorer.py  
│       └── 3_Admin_feedback.py  
│   └── lib/                  # no Streamlit here  
│       ├── answer.py  
│       ├── session.py  
│       └── logging.py  
├── environment.yml           # pinned packages  
├── snowflake.yml  
├── dbt/                      # Module 2  
├── eval/                    # Module 3 gate  
├── tests/                   # unit tests for lib/  
└── .github/workflows/deploy.yml
```

Thin UI, thick lib

pages only render; lib/ holds logic you can unit-test

One change, one PR

app, dbt model and prompt ship together, reviewed together

Pinned packages

environment.yml: production runs what was tested

Pages per audience

analysts chat and explore; admins review feedback

One codebase, three environments, zero hard-coded names

SNOWFLAKE CLI PROJECT

```
# app/snowflake.yml
definition_version: 2
env:
  DB: NEO_NEXUS_DEV           # overridden in CI
  WH: APP_DEV_WH
entities:
  assistant:
    type: streamlit
    identifier:
      name: neo_nexus_assistant
      database: <% ctx.env.DB %>
      schema: APPS
    query_warehouse: <% ctx.env.WH %>
    main_file: streamlit_app.py
    pages_dir: pages/
    stage: app_stage
    artifacts:
      - streamlit_app.py
      - pages/
      - lib/
      - environment.yml
```

DEV**NEO_NEXUS_DEV**
developers, sample data**TEST****NEO_NEXUS_TEST**
CI deploys; gate runs here**PROD****NEO_NEXUS_PROD**
users; deploy needs approval

LIB/SESSION.PY

```
# inside the app: never hard-code
DB = session.get_current_database()
```

Stream answers, and fail politely

PAGES/1_ASSISTANT.PY

```
with st.chat_message("assistant"):
    try:
        stream = complete(ANSWER_MODEL, prompt,
                           session=session, stream=True)
        text = st.write_stream(stream)      # words appear live
        log_turn(q, text, status="ok")
    except Exception as e:                  # timeout, throttle,
        text = ("Sorry, I can't answer that right now. "
               "Please try again, or contact the "
               "loyalty support team.")
        st.warning(text)
        log_turn(q, None, status="error",   # counted in
                  error=type(e).__name__)  # obs.daily_health
```

Stream

First words in about a second; the full answer still takes as long, but feels fast.

Fail politely

A clear message and a human fallback; the error is logged, never shown as a stack trace.

02

Automate

Two ways to deploy, one pipeline with an evaluation gate, and a rollback you have rehearsed

Push from CI, or let Snowflake pull from Git

A • Snowflake CLI from CI

CI uploads the files and runs snow streamlit deploy. Works with any Git host and CI tool. Used in today's lab.

B • Snowflake Git repository

Snowflake holds a clone of the repository; the app is created from a branch or tag, refreshed with FETCH. Private repositories need GIT_CREDENTIALS pointing at a secret.

SNOWFLAKE SQL

```
-- path B: Snowflake pulls from Git
create api integration git_api
  api_provider = git_https_api
  api_allowed_prefixes =
    ('https://github.com/neo-nexus/')
  enabled = true;

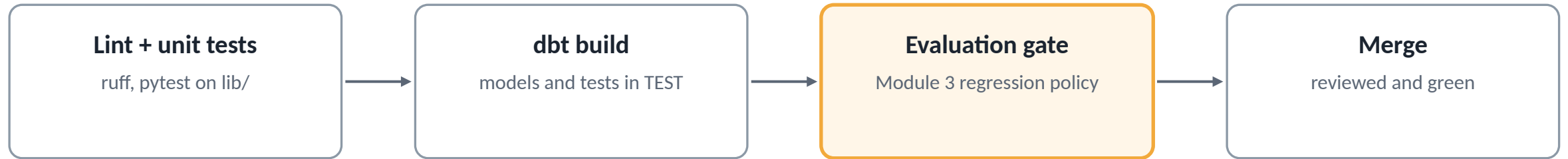
create git repository apps.assistant_repo
  api_integration = git_api
  origin =
    'https://github.com/neo-nexus/assistant.git';

alter git repository apps.assistant_repo fetch;

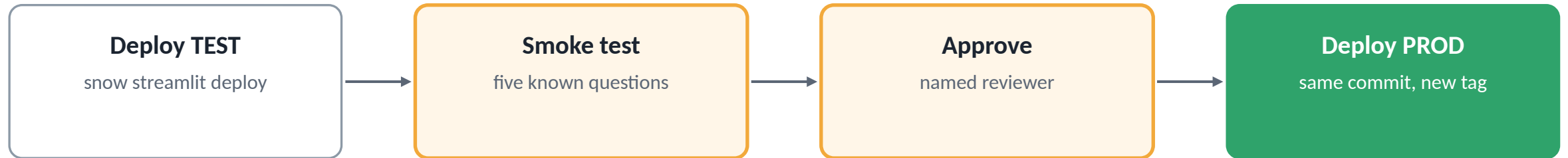
create or replace streamlit apps.neo_nexus_assistant
  from @apps.assistant_repo/branches/main/app
  main_file      = 'streamlit_app.py'
  query_warehouse = app_prod_wh;
```

Every change passes the same gates, in the same order

ON PULL REQUEST



ON MERGE TO MAIN



Build once, promote many: the commit that passed the gates is exactly the one users get.

The pipeline as code: GitHub Actions and the Snowflake CLI

`.GITHUB/WORKFLOWS/DEPLOY.YML`

```
name: deploy-assistant
on:
  push:
    branches: [main]
permissions:
  id-token: write          # OIDC, no stored keys
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: prod      # requires an approver
    steps:
      - uses: actions/checkout@v4
      - uses: snowflakedb/snowflake-actions@v3
        with:
          use-oidc: true
      - name: Evaluation gate
        run: python eval/run_gate.py --target PROD
      - name: Deploy app
        working-directory: app
        env: { DB: NEO_NEXUS_PROD, WH: APP_PROD_WH }
        run: snow streamlit deploy --replace
```

Official action

snowflakedb/snowflake-actions installs the CLI

OIDC, not secrets

no long-lived Snowflake credentials in GitHub

Approval gate

the prod environment needs a named reviewer

Same file, all envs

only DB and WH change

Module 3's regression policy, as a script that can say no

PYTHON

```
# eval/run_gate.py
results = run_eval_set(session, target=args.target) # Module 3
scores = judge(session, results) # per slice
base = session.table("eval.baseline").to_pandas()

drop = (base.set_index("slice")[METRICS]
        - scores.set_index("slice")[METRICS])

failed = []
if (drop > 0.02).any().any(): # scores 0 to 1
    failed.append(f"regression:\n{drop.round(3)}")
if scores["adversarial_pass"].min() < 1.0:
    failed.append("adversarial case failed")
if scores["cost_per_answer"].mean() > 1.10 * base_cost:
    failed.append("cost up more than 10%")

if failed:
    print("\n".join(failed)); sys.exit(1) # block deploy
save_candidate_scores(session, scores)
```

Quality

no slice drops more than 0.02

Safety

every adversarial case passes

Cost

cost per answer rises no more than 10%

Know how to undo every kind of change before you ship it

What changed	How to roll back	Speed
App code	redeploy the previous Git tag through the pipeline	minutes
Prompt version	set the previous version active in app.prompts	seconds
Model choice	change the model string in the config table	seconds
dbt models / data	revert the PR and rebuild; Time Travel for data	minutes to hours
Search service	keep the old service; switch the app back to it	seconds, if prepared

Design rule: prompts and model choices are configuration, not code, so the fastest rollbacks need no deployment.

03

Secure

Least-privilege roles for the app, its users, its admins and the pipeline, and protecting the data in conversations

Four roles, each with only what its job needs

APP_OWNER

owns the app; reads marts, uses search and semantic view, writes obs

CRM_ANALYST

USAGE on the app; nothing else

ASSISTANT_ADMIN

reads obs for the admin feedback page

CI_DEPLOYER

creates and replaces apps in APPS; no data access

SNOWFLAKE SQL

```
-- the app's owner: exactly what it needs
grant usage on cortex search service
    serving.loyalty_search      to role app_owner;
grant select on semantic view
    serving.rewards_sv         to role app_owner;
grant select on all tables in schema marts
                                to role app_owner;
grant insert on all tables in schema obs
                                to role app_owner;

-- users: open the app, nothing more
grant usage on streamlit apps.neo_nexus_assistant
                                to role crm_analyst;

-- the pipeline: deploy, never read data
grant create streamlit on schema apps
                                to role ci_deployer;
```

Conversations are customer data: treat them that way



Mask free text

masking policy on question and answer columns; only admins see raw text



Retain, then delete

a scheduled task removes turns older than the agreed retention period



No outbound access

no external access integration: data cannot leave through the app



Audit

query and access history show who read which conversations

04

Operate

Health views, alerts, cost controls and a runbook: what keeps the assistant good after launch day

One view answers: is the assistant healthy today?

SNOWFLAKE SQL

```
create or replace view obs.daily_health as
select
  date_trunc('day', t.asked_at)           as day,
  count(*)                             as turns,
  count(distinct t.session_id)          as sessions,
  avg(iff(f.rating = -1, 1, 0))          as thumbs_down_rate,
  avg(iff(t.abstained, 1, 0))            as abstain_rate,
  avg(iff(t.status = 'error', 1, 0))     as error_rate,
  approx_percentile(t.latency_ms, 0.95) as p95_latency_ms
from obs.chat_turns t
left join obs.feedback f using (turn_id)
group by day;
```

Volume turns, sessions

Quality thumbs down, abstains

Speed p95 latency

Reliability error rate

Cost per answer, from usage views

abstained and status are two columns added to obs.chat_turns in this module.

Let Snowflake watch the assistant for you

ALERT

```
create or replace alert obs.quality_alert
warehouse = ops_wh
schedule = '60 minute'
if (exists (select 1 from obs.daily_health
            where day = current_date()
            and thumbs_down_rate > 0.15))
then call system$send_email('ops_email_int',
                            'ai-team@neonexus.example',
                            'Assistant quality alert',
                            'Thumbs-down rate above 15% today');
alter alert obs.quality_alert resume;
```

RESOURCE MONITOR

```
create resource monitor app_wh_monitor
with credit_quota = 100 frequency = monthly
start_timestamp = immediately
triggers on 80 percent do notify
on 100 percent do suspend;
alter warehouse app_prod_wh set resource_monitor = app_wh_monitor;
```

Quality and reliability

Alerts on thumbs-down rate, error rate and p95 latency, each against a threshold set from the baseline.

Cost

Resource monitors cap warehouses; budgets cover AI tokens and search serving.

When something goes wrong, start here

Symptom	First check	Usual fix
Answers ignore new programme terms	dbt run status; search service refresh lag	rerun dbt; check change tracking and TARGET_LAG
Assistant is slow	p95 latency; warehouse queueing	size the query warehouse; smaller model for routing
Errors spike	status and error columns in obs.chat_turns	model deprecated or throttled: switch model config
Thumbs-down spike	failure classification (Module 3)	fix the rung the failures point to
Cost spike	Cortex usage views by user and model	cap context, cache, tier models; set budgets

Rehearse it: a runbook nobody has practised is a document, not a capability.

Twelve checks before the first real user

Build

- ☐ Code in Git, reviewed
- ☐ Pinned packages
- ☐ DEV, TEST, PROD from one codebase
- ☐ Pipeline deploys; nobody deploys by hand

Quality and security

- ☐ Evaluation gate passing on every slice
- ☐ Adversarial cases all pass
- ☐ Least-privilege roles granted
- ☐ Log masking and retention in place

Operations

- ☐ Health view on the admin page
- ☐ Alerts routed to a named owner
- ☐ Resource monitor and budgets set
- ☐ Rollback rehearsed

Take the Neo Nexus assistant to production

- | | | |
|----------|---|--------|
| 1 | Repository
restructure into app/, lib/, pages/; add snowflake.yml | 15 min |
| 2 | Pipeline
workflow deploys to TEST with the Snowflake CLI | 20 min |
| 3 | Gate
wire run_gate.py; push a worse prompt; watch it fail | 20 min |
| 4 | Roles
create the four roles; test as CRM_ANALYST | 15 min |
| 5 | Operate
health view, one alert, then a timed rollback drill | 20 min |

Deliverables

- A green pipeline run to TEST, and a red one blocked by the gate
- Four roles, with a screenshot of what CRM_ANALYST can see
- obs.daily_health and one resumed alert
- Rollback drill: what you rolled back, and how long it took

Stretch: switch deployment to the Snowflake Git repository path

Questions worth arguing about

1. A change passes every gate and still makes users unhappy. What did the gate miss, and how do you catch it next time?
2. Should a business user be allowed to edit a prompt in production without a pull request?
3. Who should be paged at 2 a.m. when the assistant starts giving wrong answers?
4. How does this pipeline change when the assistant also answers in Slack and other apps?



References

Snowflake docs: Create and deploy Streamlit apps using Snowflake CLI

Snowflake docs: Snowflake CLI GitHub Action

Snowflake docs: CREATE GIT REPOSITORY; Git integration

Snowflake docs: Alerts; resource monitors; budgets

Google SRE book: Monitoring distributed systems; postmortem culture

SnowPro prep: access control, resource monitors and alerts topics

Cortex beyond Snowflake



REST APIs

call Cortex from any service, language or platform



Agents and MCP

Cortex Agents as tools for other AI systems, and the reverse



Where people work

Slack, Teams, BI tools and custom web front ends

Before Module 6: create a programmatic access token for your user, and have Slack or Teams admin access in a test workspace if you can.

Cortex Beyond Snowflake: Integrating AI Across Platforms

Taking the governed assistant to other apps, other AI systems and the tools people already use: REST APIs, Cortex Agents, MCP, Microsoft Teams and Slack

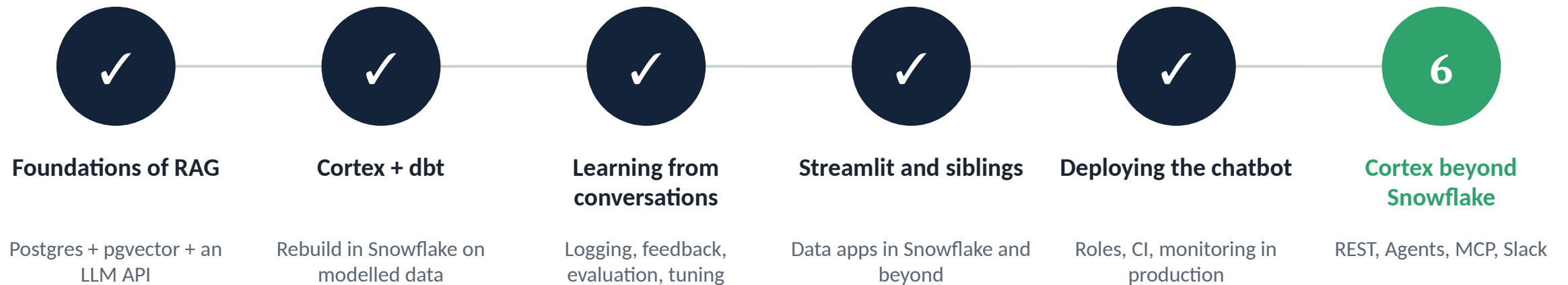
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



The same brain, reachable from everywhere



Today's output: a Cortex Agent, a REST client, a Snowflake-managed MCP server, and the assistant answering in Slack or Teams.

By the end of today you can...



Expose

Cortex Search, Analyst, LLMs and agents over REST, with the right authentication for each caller.



Orchestrate

a Cortex Agent that plans across search, analytics and custom tools, replacing the hand-built router.



Connect

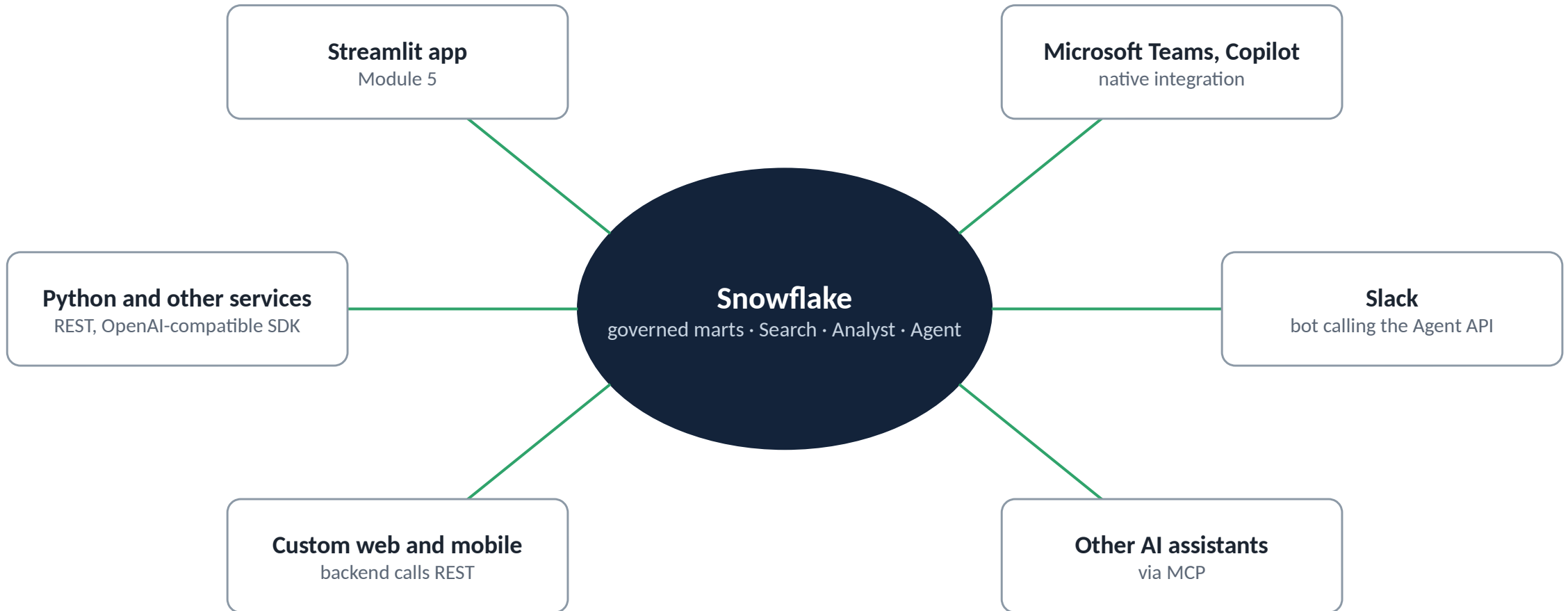
other AI assistants to governed Snowflake data through a Snowflake-managed MCP server.



Govern

every surface, deciding whose identity and permissions each answer runs under.

Move the question to the data, not the data to the question



One governed hub; every surface sends questions in and gets answers out.

Pick the interface by who is calling

Interface	Who calls it	What they get	Typical use
SQL functions	dbt, BI tools, notebooks, any SQL client	AI_COMPLETE, AI_CLASSIFY, SEARCH_PREVIEW, semantic views	Batch enrichment, analytics
REST APIs	Services in any language	Search, Analyst, chat completions, Agents	Custom apps, Slack bots, back ends
MCP server	AI assistants and agent frameworks	Discoverable tools: agent, search, analyst, SQL	Claude, Cursor, other agents
Native integrations	End users	Agents inside Teams and Copilot; Snowflake Intelligence	Business users, no code

Same governed objects behind all four: one search service, one semantic view, one agent.

01

REST APIs

Calling Cortex from any language or platform, and choosing the right way to authenticate

Every capability we built has an HTTP endpoint

Capability	Endpoint (under https://<account>.snowflakecomputing.com)
Chat completions	/api/v2/cortex/v1/chat/completions
Cortex Search	/api/v2/databases/{db}/schemas/{schema}/cortex-search-services/{name}:query
Cortex Analyst	/api/v2/cortex/analyst/message
Cortex Agent (object)	/api/v2/databases/{db}/schemas/{schema}/agents/{name}:run
MCP server	/api/v2/databases/{db}/schemas/{schema}/mcp-servers/{name}

Objects, not endpoints, carry the permissions: grant USAGE on the service or agent, and the API follows.

Choose authentication by whose identity the call carries



Programmatic access token

Simple bearer token for a user. Restrict to a least-privileged role and set an expiry.

Prototypes, MCP clients



OAuth

Carries the end user's identity: their role, row access and masking policies apply.

Acting for a person



Key-pair JWT

A service user signs requests with a private key held in a secrets manager.

Back-end services and jobs

Never in a browser: front ends call your back end; only the back end holds Snowflake credentials.

If it can send HTTPS, it can use the assistant

BASH • CORTEX SEARCH REST

```
curl -X POST \
  "https://$ACCOUNT.snowflakecomputing.com/api/v2/databases/NEO_NEXUS/schemas/SERVING/cortex-search-services/LOYALTY_SEARCH:query" \
  -H "Authorization: Bearer $SNOWFLAKE_PAT" \
  -H "Content-Type: application/json" \
  -d '{"query": "can I redeem a free drink on delivery",
      "columns": ["chunk_id", "chunk_text"],
      "filter": {"@eq": {"channel": "App"}},
      "limit": 5}'
```

Java, Node, Go

same JSON over HTTPS

Workflow tools

UiPath, Power Automate, n8n HTTP steps

Existing LLM code

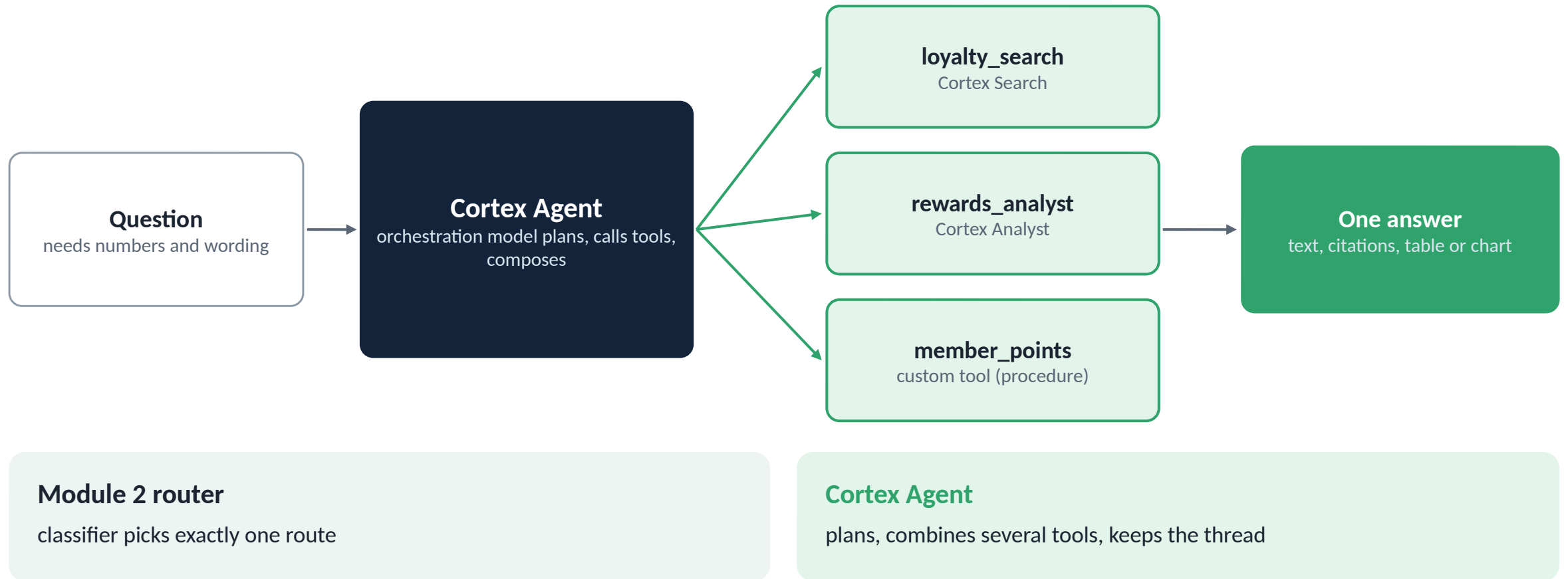
OpenAI SDK with a Snowflake base URL

02

Cortex Agents

From our hand-built router to a managed agent that plans, calls tools and streams its reasoning

An agent plans which tools to use, then answers



The agent is a governed object, defined in YAML

SNOWFLAKE SQL

```
create or replace agent serving.neo_nexus_agent
from specification $$
models:
  orchestration: claude-sonnet-4-5
instructions:
  response: "Answer concisely. Cite programme chunks.
    Say when you are not sure."
  orchestration: "Use loyalty_search for terms and
    FAQs; rewards_analyst for counts and trends."
tools:
  - tool_spec: { type: cortex_search, name: loyalty_search }
  - tool_spec: { type: cortex_analyst_text_to_sql,
    name: rewards_analyst }
tool_resources:
  loyalty_search:
    name: NEO_NEXUS.SERVING.LOYALTY_SEARCH
    max_results: 5
  rewards_analyst:
    semantic_view: NEO_NEXUS.SERVING.REWARDS_SV
    execution_environment:
      { type: warehouse, warehouse: APP_PROD_WH }
$$;
```

Models

which LLM orchestrates

Instructions

how to respond; how to choose tools

Tools

what the agent may use

Tool resources

the real objects behind each tool

One streaming endpoint serves every surface

PYTHON

```
def run_agent(question, token):
    url = (f"https://{ACCOUNT}.snowflakecomputing.com/api/v2"
          "/databases/NEO_NEXUS/schemas/SERVING"
          "/agents/NEO_NEXUS_AGENT:run")
    body = {"messages": [{"role": "user", "content": [
        {"type": "text", "text": question}]}]}
    headers = {"Authorization": f"Bearer {token}",
              "Accept": "text/event-stream"}
    answer, event = [], None
    with requests.post(url, json=body, headers=headers,
                      stream=True) as r:
        for line in r.iter_lines(decode_unicode=True):
            if line.startswith("event:"):
                event = line[6:].strip()
            elif line.startswith("data:") and \
                 event == "response.text.delta":
                answer.append(json.loads(line[5:])["text"])
    return "".join(answer)
```

response.text.delta

answer, as it streams

response.tool_use

which tool, which input

response.table

SQL results to render

response

final aggregated answer

What the agent adds, and what it costs

	Module 2 router	Cortex Agent
Multi-part questions	One route per question	Plans across several tools
Outputs	Text	Text, tables, charts, citations
Conversation state	Your own session state	Threads managed by the API
Extra tools	More code per tool	Declare a tool in the spec
Latency and cost	One classification call	Orchestration on every question
Predictability	Explicit rule, easy to test	Model-planned; test with evals

Decide with the Module 5 gate: run both on the evaluation set and compare quality, latency and cost.

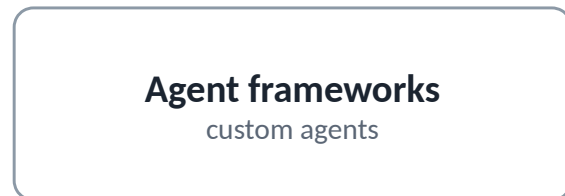
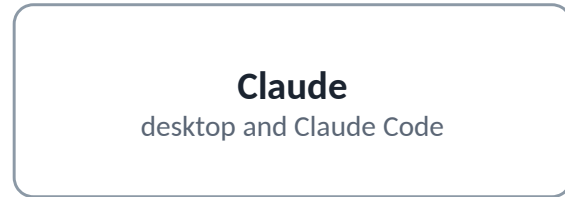
03

MCP

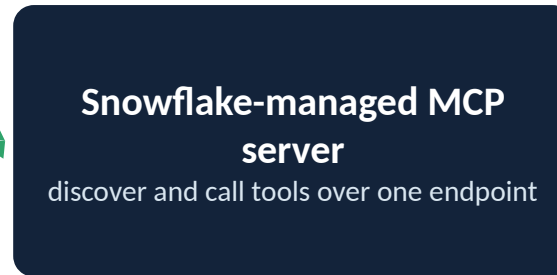
The Model Context Protocol: letting other AI assistants use Snowflake's governed tools

A common plug between AI assistants and tools

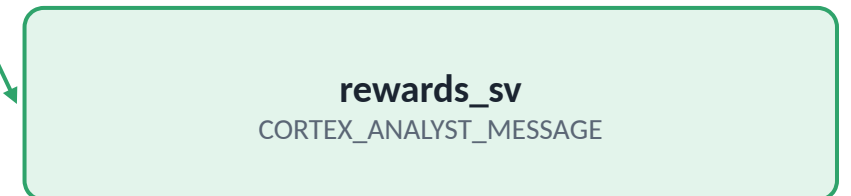
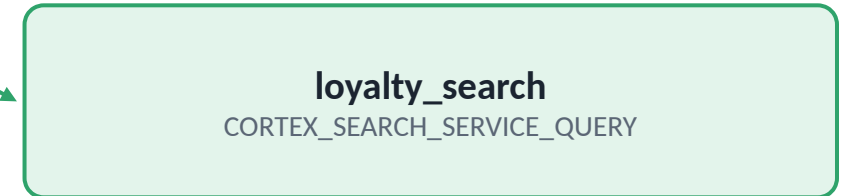
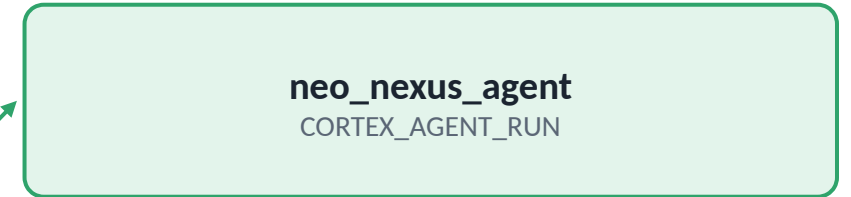
MCP CLIENTS



PROTOCOL



GOVERNED TOOLS



Before MCP: one custom integration per assistant per system. **With MCP:** one server, any compliant client.

One statement publishes governed tools to any MCP client

SNOWFLAKE SQL

```
create or replace mcp server serving.neo_nexus_mcp
from specification $$
tools:
  - name: "neo-nexus-assistant"
    type: "CORTEX_AGENT_RUN"
    identifier: "NEO_NEXUS.SERVING.NEO_NEXUS_AGENT"
    title: "Neo Nexus loyalty assistant"
    description: "Answers questions about Caffè Nero loyalty
      terms and rewards statistics, with citations"
  - name: "loyalty-search"
    type: "CORTEX_SEARCH_SERVICE_QUERY"
    identifier: "NEO_NEXUS.SERVING.LOYALTY_SEARCH"
    title: "Loyalty terms search"
    description: "Searches loyalty terms, FAQs and store guides"
$$;
```

```
https://<account>.snowflakecomputing.com/api/v2/databases/NEO_NEXUS/schemas/SERVING/
mcp-servers/NEO_NEXUS_MCP
```

Tools point at objects

the agent and search service from earlier; nothing new to maintain

Write descriptions for a model

the client's model chooses tools from their descriptions: be specific about what each answers

An MCP server is a door: grant it like one

SNOWFLAKE SQL

```
create role mcp_neo_nexus;  
grant database role snowflake.cortex_agent_user  
  to role mcp_neo_nexus;  
grant usage on warehouse app_prod_wh to role mcp_neo_nexus;  
grant usage on database neo_nexus      to role mcp_neo_nexus;  
grant usage on schema neo_nexus.serving  
  to role mcp_neo_nexus;  
grant usage on mcp server neo_nexus.serving.neo_nexus_mcp  
  to role mcp_neo_nexus;  
grant usage on agent neo_nexus.serving.neo_nexus_agent  
  to role mcp_neo_nexus;  
grant usage on cortex search service  
  neo_nexus.serving.loyalty_search      to role mcp_neo_nexus;
```

OAuth by default

each user's identity and policies flow through

Least-privileged tokens

if you must use a PAT, restrict its role

Read-only SQL, if any

read_only: true and a query timeout

Vet clients and servers

tool text is a prompt-injection surface

04

Where people work

Microsoft Teams and Copilot, Slack, custom front ends, and the identity question every surface raises

Three routes to the people who need answers



Teams and Copilot

Snowflake-provided integration for Cortex Agents. Admin setup; no code to host.

Lowest effort



Slack

A small Bolt bot calls the Agents Run API. Snowflake publishes a quickstart.

Hosting a bot



Custom web and mobile

Your front end calls your back end; only the back end calls the agent.

Full control

All three call the same agent object, under the same grants.

Mention the bot, get a governed answer in the thread

PYTHON • SLACK BOLT

```
import os
from slack_bolt import App
from slack_bolt.adapter.socket_mode import SocketModeHandler

app = App(token=os.environ["SLACK_BOT_TOKEN"])

@app.event("app_mention")
def on_mention(event, say):
    question = event["text"].split(">", 1)[-1].strip()
    thread = event.get("thread_ts", event["ts"])
    say(text="Checking the loyalty library...", thread_ts=thread)
    token = token_for(event["user"])      # whose identity?
    answer = run_agent(question, token)    # Part 2
    log_turn(question, answer, surface="slack")
    say(text=answer, thread_ts=thread)

SocketModeHandler(app, os.environ["SLACK_APP_TOKEN"]).start()
```

Threads

answers stay in a thread, channels stay readable

Logged like any turn

surface = 'slack' in obs.chat_turns

The open question

token_for(user): whose permissions?

Whose permissions does each answer run under?

Pattern	How it works	Policies per person?	Use when
Service identity	one service user for all requests	No	public or non-sensitive data only
Mapped roles	chat groups map to Snowflake roles	Per group	coarse, team-level access is enough
Per-user OAuth	each person signs in; their token is used	Yes	sensitive data, row-level rules, audit

Rule for personal data: if row access policies matter, the person's own identity must reach Snowflake. Anything else is a quiet bypass of your governance.

Cortex plugs into the tools developers already use



Existing LLM code

OpenAI-compatible chat completions: change the key and base URL (Module 2).



Agent frameworks

LangChain and LlamaIndex offer Snowflake integrations for models and retrieval.



BI and notebooks

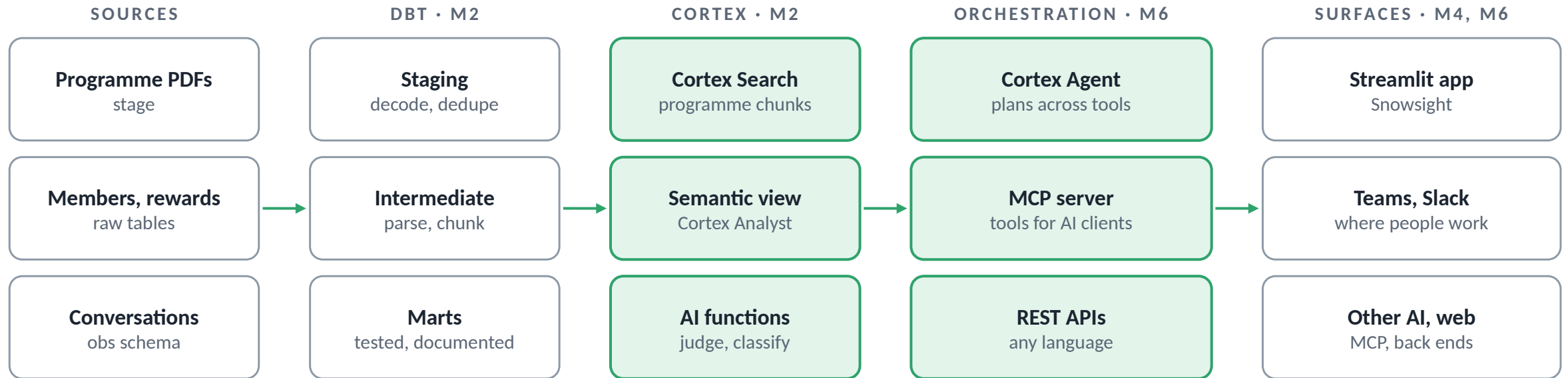
Call AI SQL functions and query semantic views from any SQL client.



Calling out

External access integrations let Snowflake call outside APIs: govern them as egress.

Reference architecture for a governed enterprise assistant



Learn • M3 log every turn • feedback • evaluation set • LLM judge • failure classification • optimisation ladder

Ship • M5 Git • CI • evaluation gate • DEV, TEST, PROD • least-privilege roles • health view, alerts, runbook

Take the assistant beyond Snowflake

- 1 Create the agent**
neo_nexus_agent with search and analyst tools
20 min
- 2 Call it over REST**
run_agent() with streaming; log tool events
15 min
- 3 MCP server**
create, grant, connect an MCP client, ask a question
20 min
- 4 Slack bot**
Bolt bot in a test workspace answering in threads
20 min
- 5 Identity test**
two users, same question: OAuth vs shared token
15 min

Deliverables

- Agent answering a question that needs both search and analyst
- An MCP client using the Snowflake MCP server
- Slack bot answering in a thread, logged to obs
- Identity test result: which pattern respected row access

Stretch: connect the agent to Microsoft Teams with the Snowflake integration

Build a governed assistant for a domain of your choice

Component	Evidence	Weight
Governed data (M2)	dbt project with tests; search service; semantic view	20%
Assistant (M2, M6)	router or agent, justified; cites sources; abstains	15%
Evaluation (M3)	eval set with slices; judge calibration; one measured improvement	20%
App and surfaces (M4, M6)	Streamlit app plus one external surface	15%
Production (M5)	pipeline with gate; roles; health view; rollback	15%
Decision record and demo	ADR on key trade-offs; 10-minute live demo	15%

Suggested weights; adapt to your programme's assessment rules.

Questions worth arguing about

1. A business owner wants the Slack bot live next week on a shared token. What do you say, and what would you accept?
2. What new risks appear when other AI assistants can call your tools through MCP?
3. After six modules, when would you still build the Module 1 DIY stack instead of Cortex?
4. Which single design decision in your capstone would be most expensive to reverse?



References

Snowflake docs: Cortex Agents; Cortex Agents Run API

Snowflake docs: Snowflake-managed MCP server

Snowflake docs: Cortex Agents for Microsoft Teams and Microsoft 365 Copilot

Snowflake docs: Cortex REST API; authentication and programmatic access tokens

Model Context Protocol: specification and documentation, modelcontextprotocol.io

Slack: Bolt for Python documentation

From a vector database to a governed AI platform



The lasting lesson: models change every few months. Governed data, honest evaluation and disciplined delivery are what make an assistant worth trusting.